

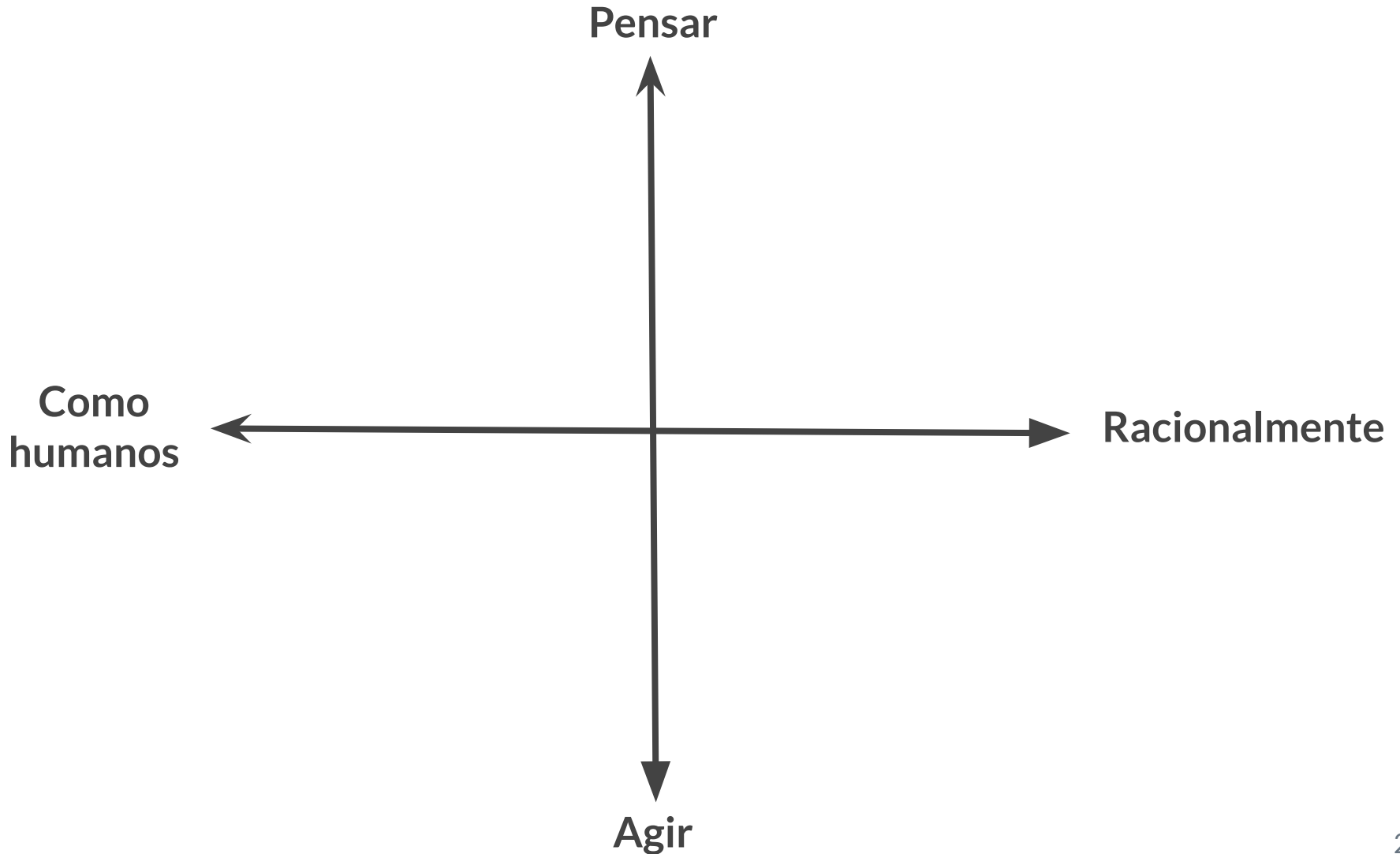
Estratégias de Busca sem Informação



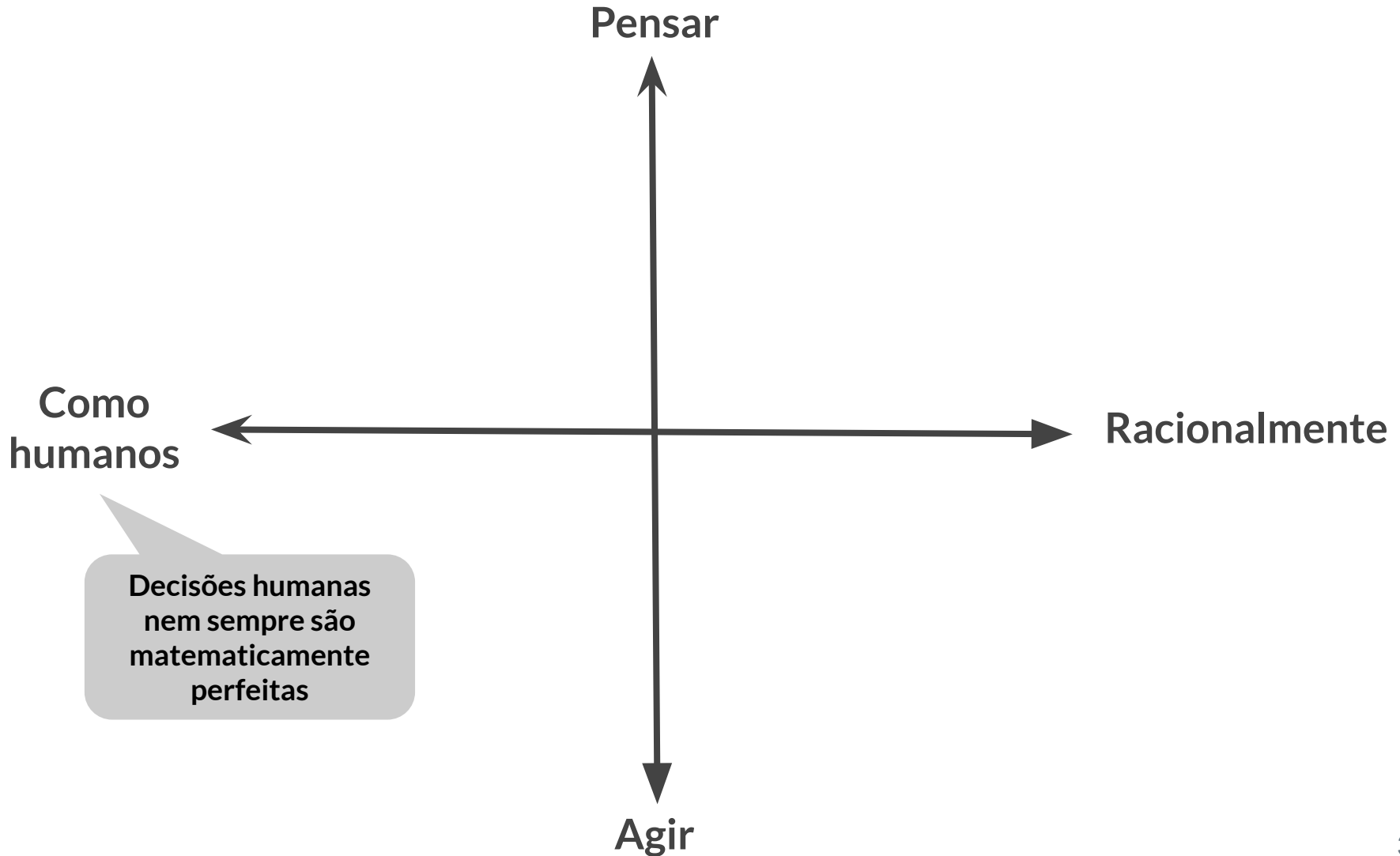
Gabriel P. Oliveira

Inteligência Artificial
2026/1

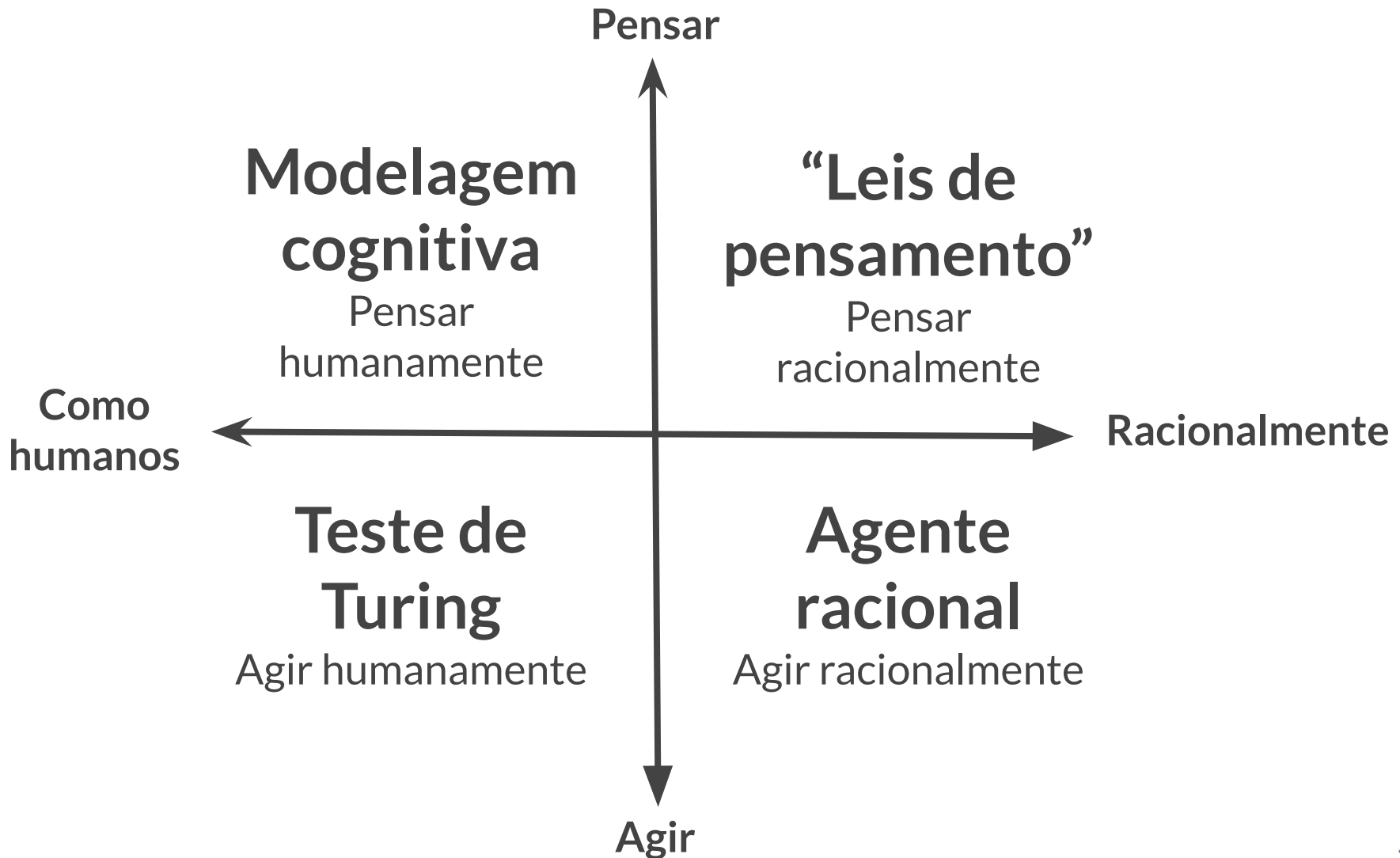
As dimensões da IA



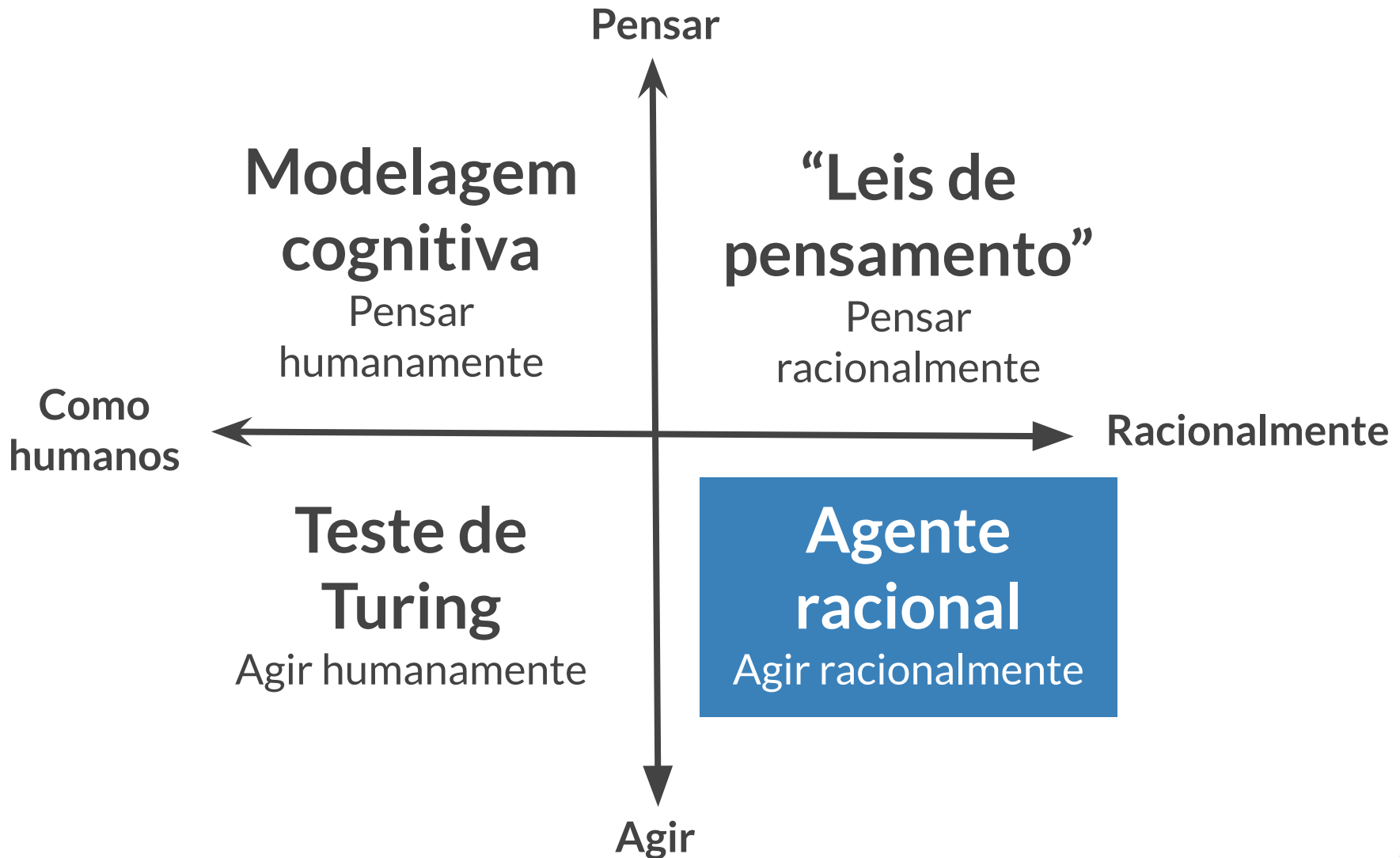
As dimensões da IA



As dimensões da IA



As dimensões da IA



Agir racionalmente

Agentes → entidades que **percebem o ambiente** e **tomam decisões (ações)** a partir dele

Racionais → *buscam sempre a melhor saída (esperada)*

Exemplos?

Agir racionalmente

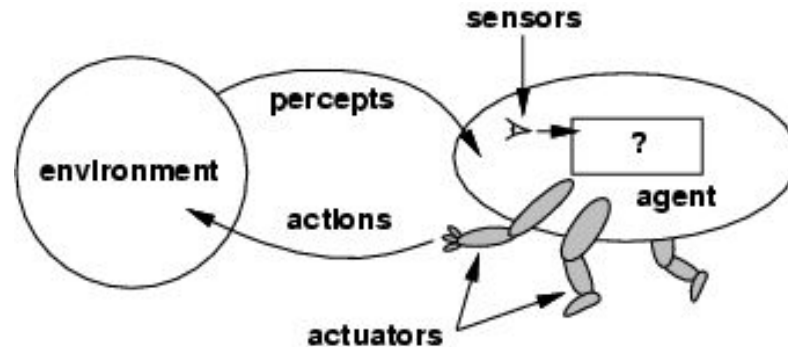
Agentes → entidades que **percebem o ambiente** e **tomam decisões (ações)** a partir dele

Racionais → *buscam sempre a melhor saída (esperada)*

Exemplos?



Agentes inteligentes



- ▷ **Percepts** → entrada em um determinado momento
- ▷ **Actions** → depende da sequência de percepts até aquele momento
- ▷ Existe uma função que faz esse “mapeamento”
- ▷ Essa função é implementada por um programa
 - **Função:** abstrata, modelo matemático do comportamento
 - **Programa:** concreto, roda na arquitetura específica

Problem-solving agents

Um tipo de agente que decide o que fazer procurando **sequências de ações que o leva a um objetivo** (estado desejado) para um dado problema

Problem-solving agents

Um tipo de agente que decide o que fazer procurando **sequências de ações que o leva a um objetivo** (estado desejado) para um dado problema

Algumas questões:

1. Como formular o problema?
2. Como definir o objetivo?
3. Como achar a solução?

Problem-solving agents

Um tipo de agente que decide o que fazer procurando **sequências de ações que o leva a um objetivo** (estado desejado) para um dado problema

Ideia:

- ▷ *A partir de um estado atual, um agente (algoritmo) pode decidir o que fazer examinando as sequências possíveis de ações que levam a estados com valores conhecidos*
- ▷ *Em seguida, basta escolher o melhor entre eles*

Problem-solving agents

Um tipo de agente que decide o que fazer procurando **sequências de ações que o leva a um objetivo** (estado desejado) para um dado problema

Como encontrar tais sequências?

Ideia:

- ▷ *A partir de um estado atual, um agente (algoritmo) pode decidir o que fazer examinando as **sequências possíveis de ações que levam a estados com valores conhecidos***
- ▷ *Em seguida, basta escolher o melhor entre eles*

Problem-solving agents

Em outras palavras:

- ▷ O objetivo é representado por um **conjunto de estados**
- ▷ O agente começa em um **estado inicial** e ações fazem o agente mudar de um estado para outro
- ▷ O agente deve encontrar então uma **sequência de ações** (de preferência a melhor) que o levem do estado inicial ao estado final

1.

Busca

*O processo mais simples para se encontrar uma dessas
sequências*

Formulação do Problema

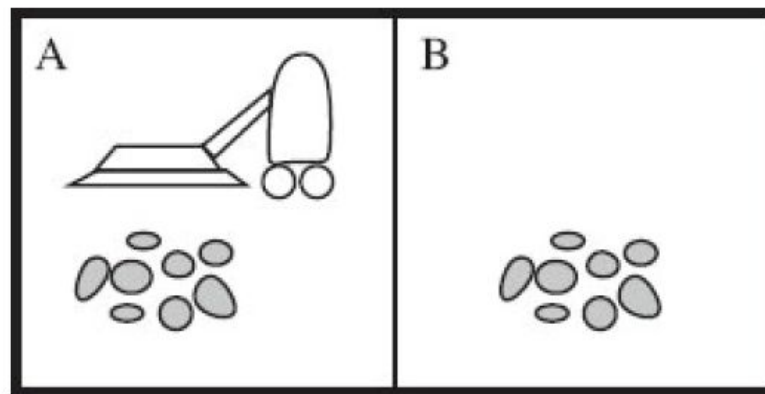
1. **Estado inicial** em que o agente começa
2. **Conjunto de ações** disponíveis para o agente
3. **Modelo de transição** (resultados das ações)
 - Função sucessora (pares <ação, estado>)
 - Estado inicial, ações e modelo de transição definem o **espaço de estados do problema**, ou seja todos os estados atingíveis a partir do estado inicial (grafo)
4. **Teste do objetivo**
 - Estado(s) explícito(s) ou propriedade abstrata (e.g., xeque-mate)
5. **Custo da solução** (do caminho)
 - Custo do caminho é a soma do custo dos passos individuais
 - Solução ótima é aquela cujo caminho tem o menor custo

Exemplo 1: Aspirador de pó

Um mundo simples com dois locais (A e B), onde **um aspirador percebe sua posição e se há sujeira**

Suas ações possíveis são **mover-se ou aspirar**

A estratégia básica é: se o local atual estiver **sujo**, ele **limpa**; se estiver **limpo**, ele se **move para o outro local**



Exemplo 1: Aspirador de pó

Estados: agente está em uma entre duas posições (A ou B) que podem estar sujas ou limpas (8 estados)

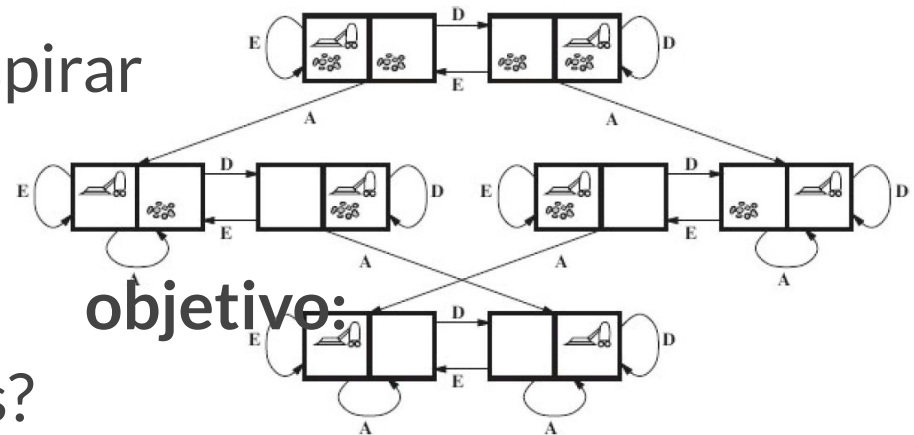
Estado inicial: qualquer um

Ações: esquerda, direita, aspirar

Modelo de transição:

Teste de
Quadrados estão limpos?

Custo: cada ação tem custo 1



Exemplo 2: 8-puzzle

Um tabuleiro 3×3 com **oito peças numeradas** e um **quadrado vazio**. Uma peça adjacente ao quadrado vazio pode deslizar para esse quadrado. O objetivo é alcançar um estado objetivo especificado.

7	2	4
5		6
8	3	1

Estado inicial

	1	2
3	4	5
6	7	8

Estado objetivo

Exemplo 2: 8-puzzle

Estados: Posição de cada uma das peças e do espaço no tabuleiro

Estado inicial: qualquer um

Ações/Modelo de transição: esquerda, direita, para cima, para baixo (*“movimento” do quadrado vazio*)

Teste de objetivo:

Verifica se a configuração foi atingida

Custo: cada ação tem custo 1

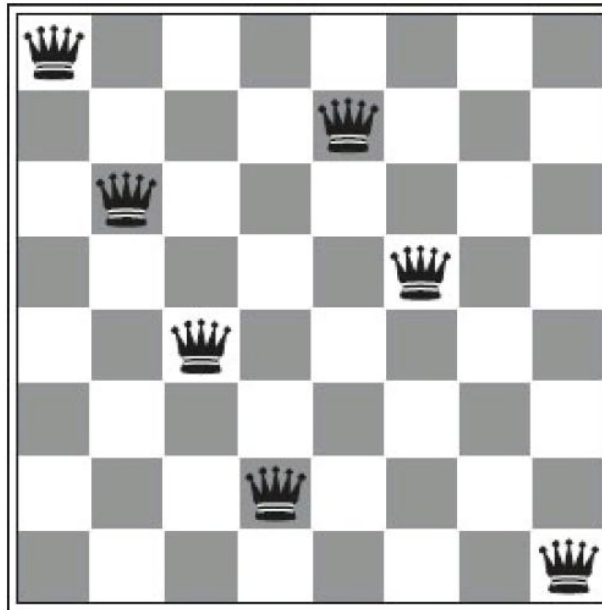
	1	2
3	4	5
6	7	8

Estado objetivo

Exemplo 3: Problema de 8 Rainhas

Posicionar oito rainhas em um tabuleiro de xadrez de tal forma que **nenhuma rainha ataque qualquer outra**

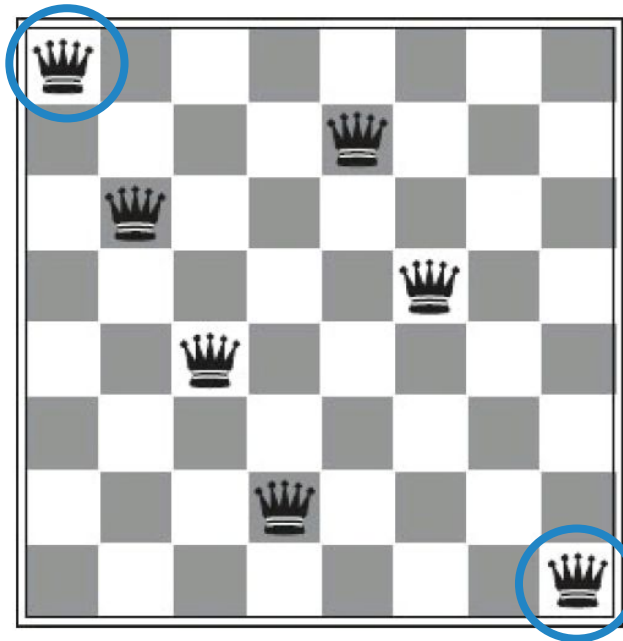
- ▷ Rainhas atacam qualquer peça na mesma linha, coluna ou diagonal



Exemplo 3: Problema de 8 Rainhas

Posicionar oito rainhas em um tabuleiro de xadrez de tal forma que **nenhuma rainha ataque qualquer outra**

- ▷ Rainhas atacam qualquer peça na mesma linha, coluna ou diagonal



Uma quase
solução :-D

Exemplo 3: Problema de 8 Rainhas

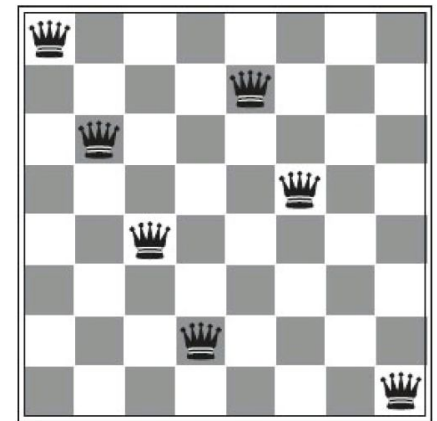
Estados: Qualquer disposição de 0-8 rainhas no tabuleiro é um estado

Estado inicial: Tabuleiro vazio

Ações/Modelo de transição: colocar uma rainha em qualquer quadrado vazio

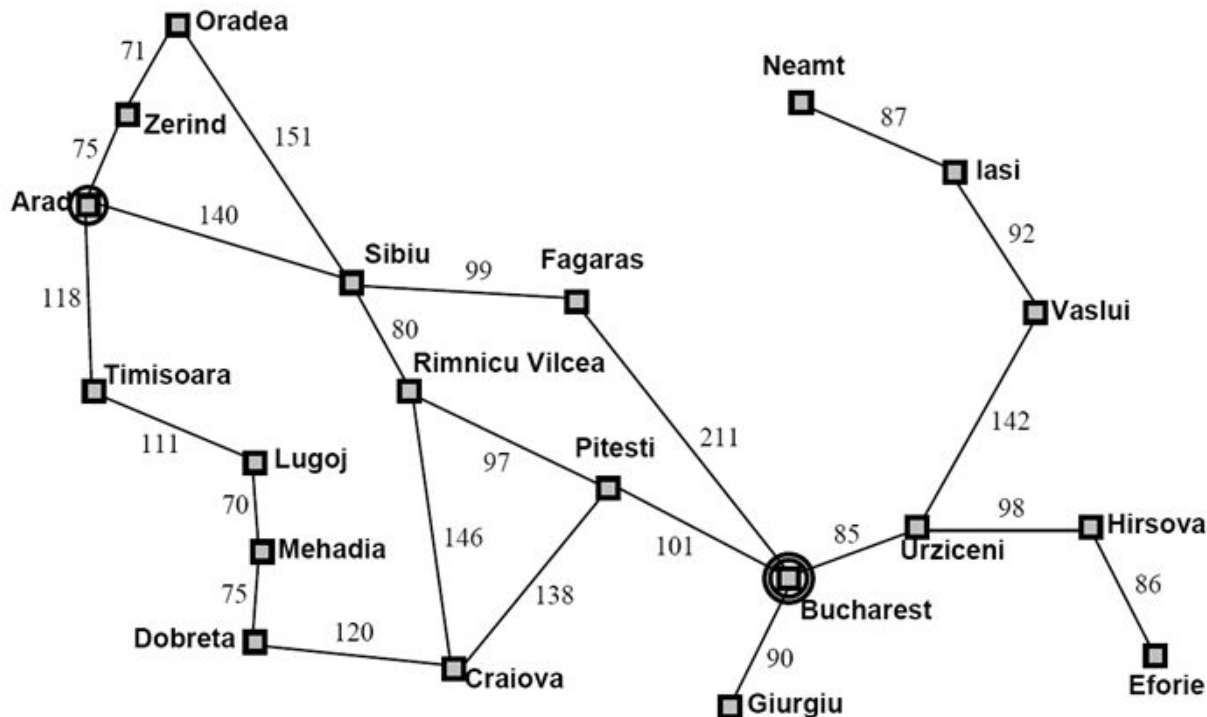
Teste de objetivo:
8 rainhas que não se atacam

Custo: o custo não importa aqui



Exemplo 4: Viagem na Romênia

Estou de férias na Romênia e devo viajar da cidade de **Arad** até a capital **Bucareste** (*Bucharest*)



Exemplo 4: Viagem na Romênia

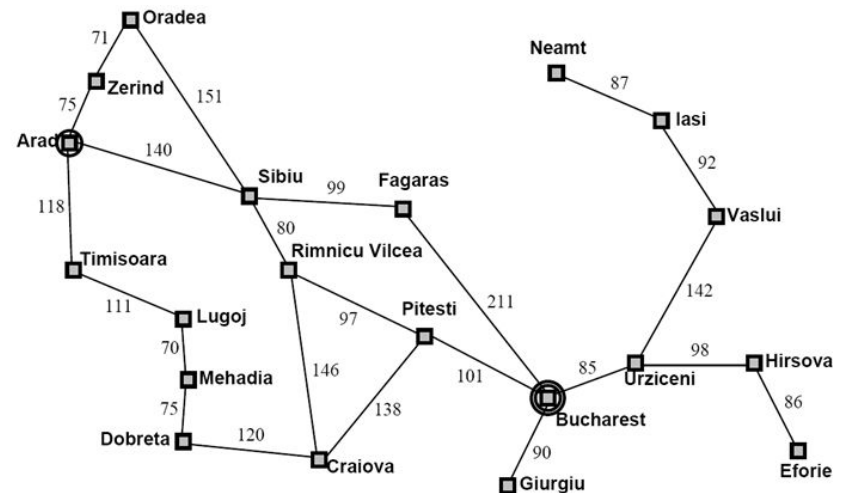
Estados: conjunto de cidades onde o agente pode estar

Estado inicial: Arad

Ações/Modelo de transição: viajar de uma cidade para outra

Teste de objetivo:
Estar em Bucareste

Custo: distância entre as cidades



Outros exemplos reais

- ▷ Problema do caixeiro viajante
- ▷ Design de circuitos VLSI
- ▷ Navegação de robôs
- ▷ Sequência automática de montagem
- ▷ ...

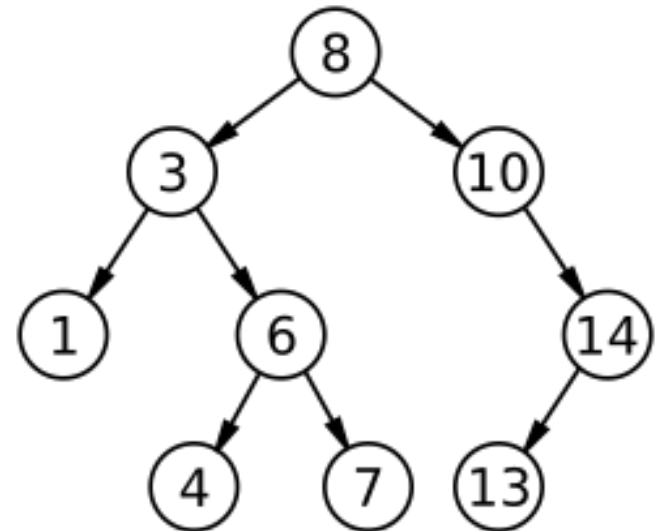
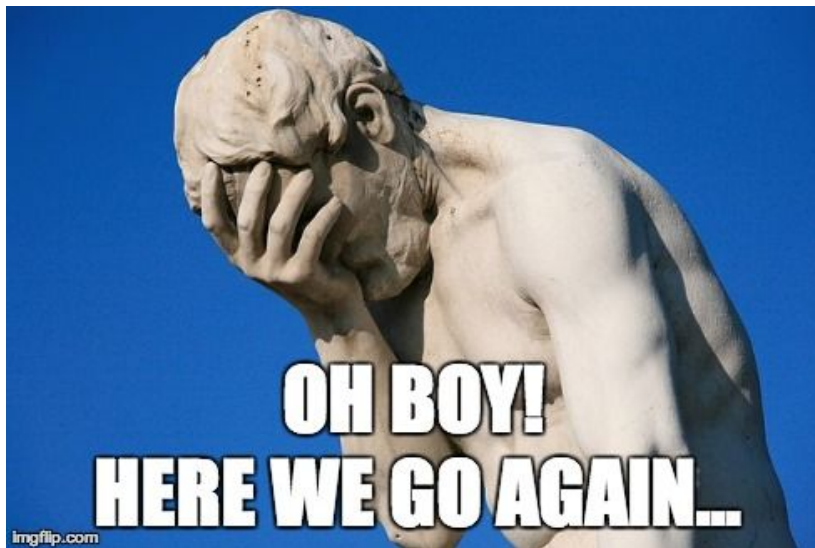
2.

Busca em Espaço de Estados

Depois de formular alguns problemas, precisamos resolvê-los

Busca em Espaço de Estados

As sequências de ações possíveis que começam a partir do estado inicial formam uma árvore de busca



Busca em Espaço de Estados

As sequências de ações possíveis que começam a partir do estado inicial formam uma árvore de busca

- ▷ **Raíz** → estado inicial
- ▷ **Arestas** → ações (alteração no ambiente)
 - Podem ter um custo ou utilidade associada
- ▷ **Nós** → representam os estados
 - Algumas folhas são estados objetivos (finais)

Busca em Espaço de Estados

As sequências de ações possíveis que começam a partir do estado inicial formam uma árvore de busca

- ▷ **Raíz** → estado inicial
- ▷ **Arestas** → ações (alteração no ambiente)
 - Podem ter um custo ou utilidade associada
- ▷ **Nós** → representam os estados
 - Algumas folhas são estados objetivos (finais)

Cada caminho a partir da raiz da árvore até outro nó representa uma **sequência de ações**

Busca em Espaço de Estados

Em outras palavras:

Navegar no espaço de busca é **construir e explorar uma árvore de busca até encontrar uma solução** ou um caminho válido ao objetivo.

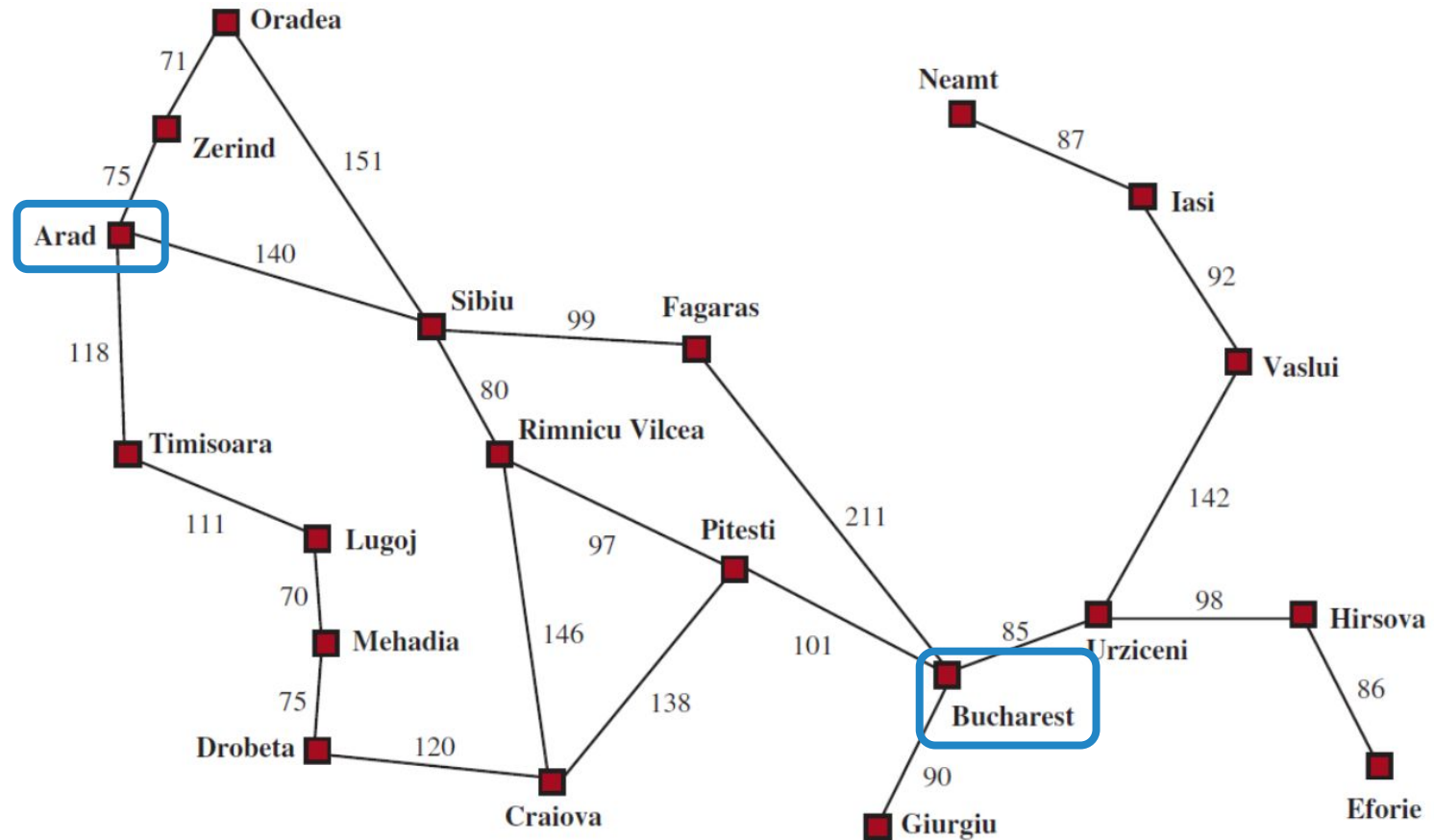
Expansão de um nó

- ▷ Aplicar as ações válidas naquele nó
- ▷ Gerar os “filhos” daquele nó na árvore
- ▷ A decisão de qual nó expandir é fundamental e define a estratégia de busca
- ▷ A expansão do nó objetivo termina a busca com sucesso

Borda (ou Fronteira) → conjunto de nós que não foram expandidos ainda

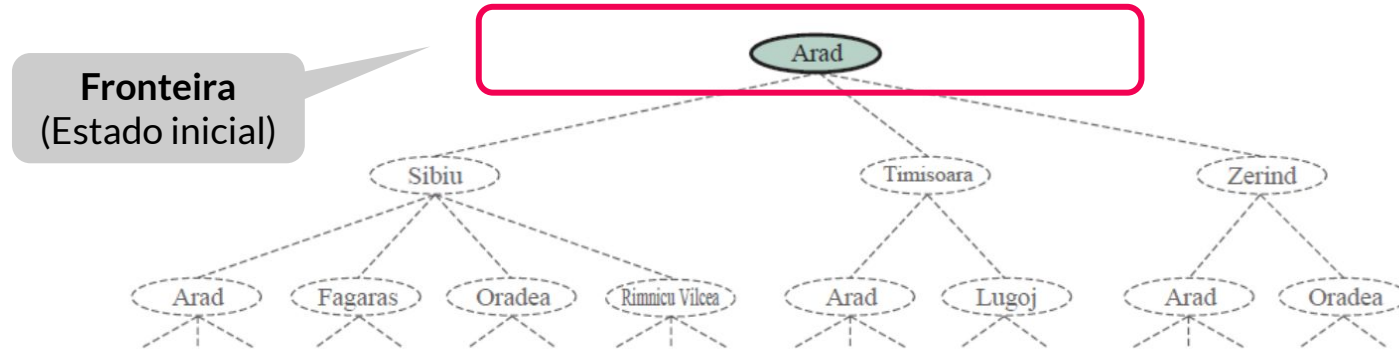
Expansão de um nó

Viagem na Romênia: Arad → Bucareste



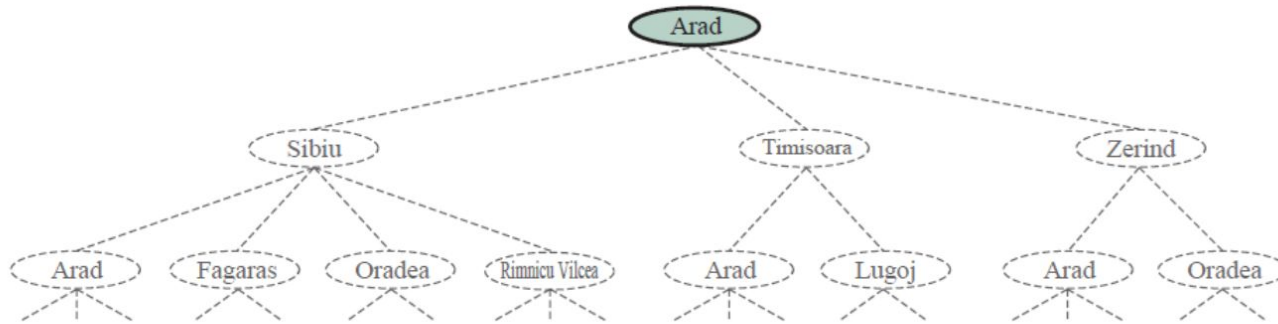
Expansão de um nó

Viagem na Romênia: Arad → Bucareste

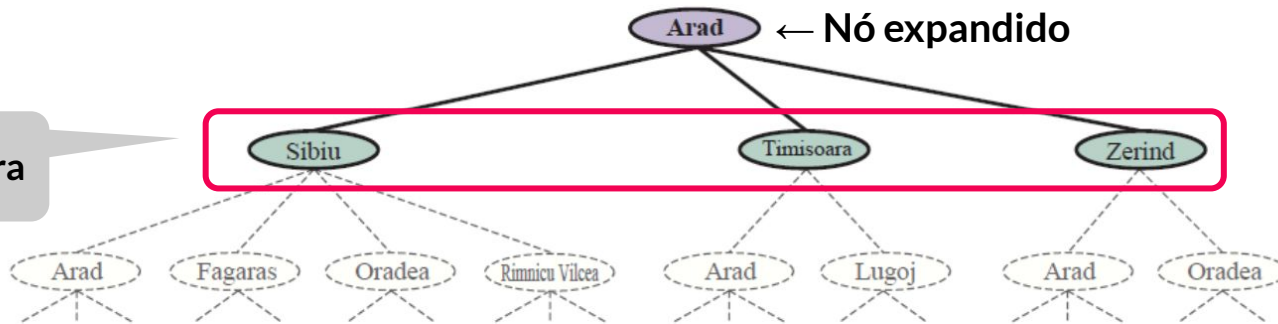


Expansão de um nó

Viagem na Romênia: Arad → Bucareste

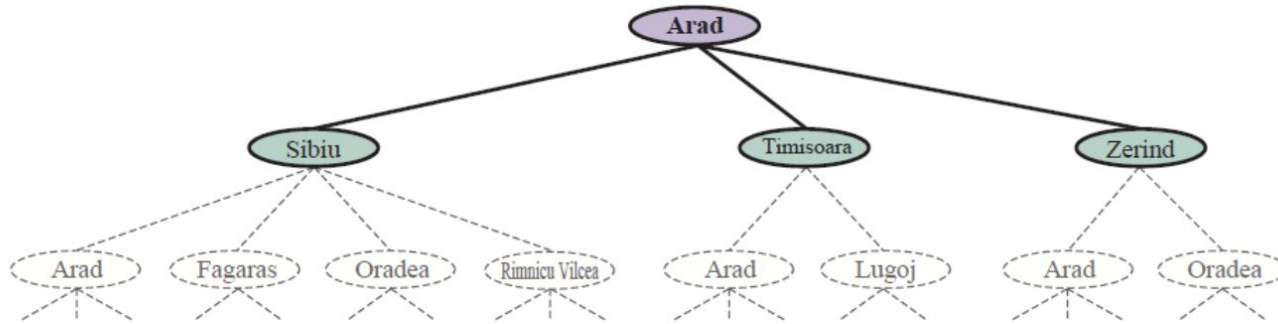
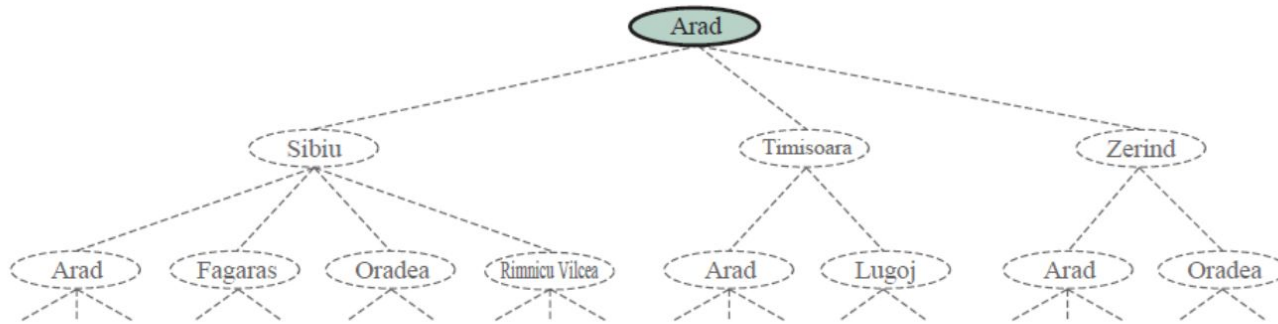


Nova fronteira



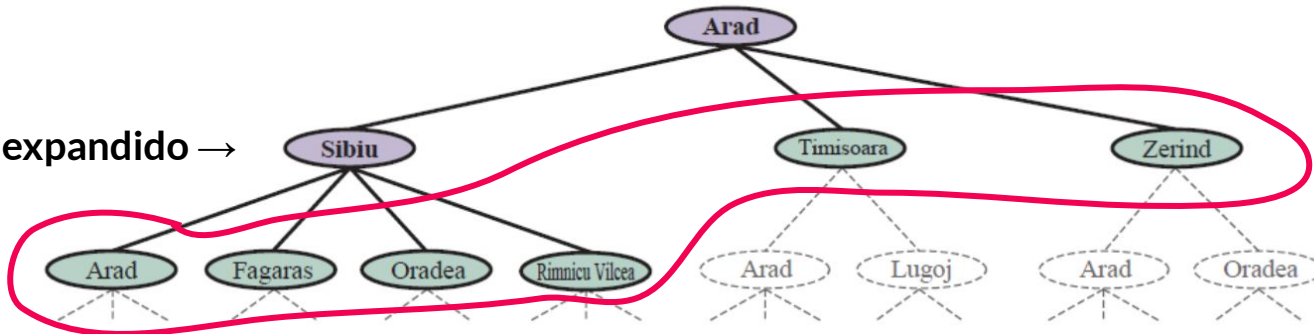
Expansão de um nó

Viagem na Romênia: Arad → Bucareste



Nó expandido →

Nova
fronteira



Avaliação dos Algoritmos

1. **Algoritmo completo:** encontra uma solução quando ela existir
2. **Algoritmo ótimo:** encontra a solução com menor custo
3. **Complexidades de Tempo e Espaço**
 - Fator de ramificação (b) → máximo de sucessores de um nó
 - Profundidade da solução mais rasa (d)
 - Comprimento máximo de qualquer caminho (m)

Avaliação dos Algoritmos

1. **Algoritmo completo:** encontra uma solução quando ela existir
2. **Algoritmo ótimo:** encontra a solução com menor custo
3. **Complexidades de Tempo e Espaço**
 - Fator de ramificação (b) → máximo de sucessores de um nó
 - Profundidade da solução mais rasa (d)
 - Comprimento máximo de qualquer caminho (m)

Custo da Busca vs. Custo Total

Custo Total = Custo de Busca + Custo da Solução

Tipos de Busca

1. Busca sem informação

- Não têm nenhuma informação adicional sobre estados além daquelas fornecidas na definição do problema

2. Busca com informação (heurística)

- Sabem se um estado não objetivo é mais “promissor”

Tipos de Busca

1. Busca sem informação

- Não têm nenhuma informação adicional sobre estados além daquelas fornecidas na definição do problema
 - *Depth-first search* (profundidade)
 - *Depth-limited search* (profundidade limitada)
 - *Iterative deepening search* (profundidade iterativa)
 - *Breadth-first search* (largura)
 - *Uniform cost search* (custo uniforme)
 - *Bidirectional search*

3.

Busca em Largura (BFS)

Breadth-first search

Busca em Largura

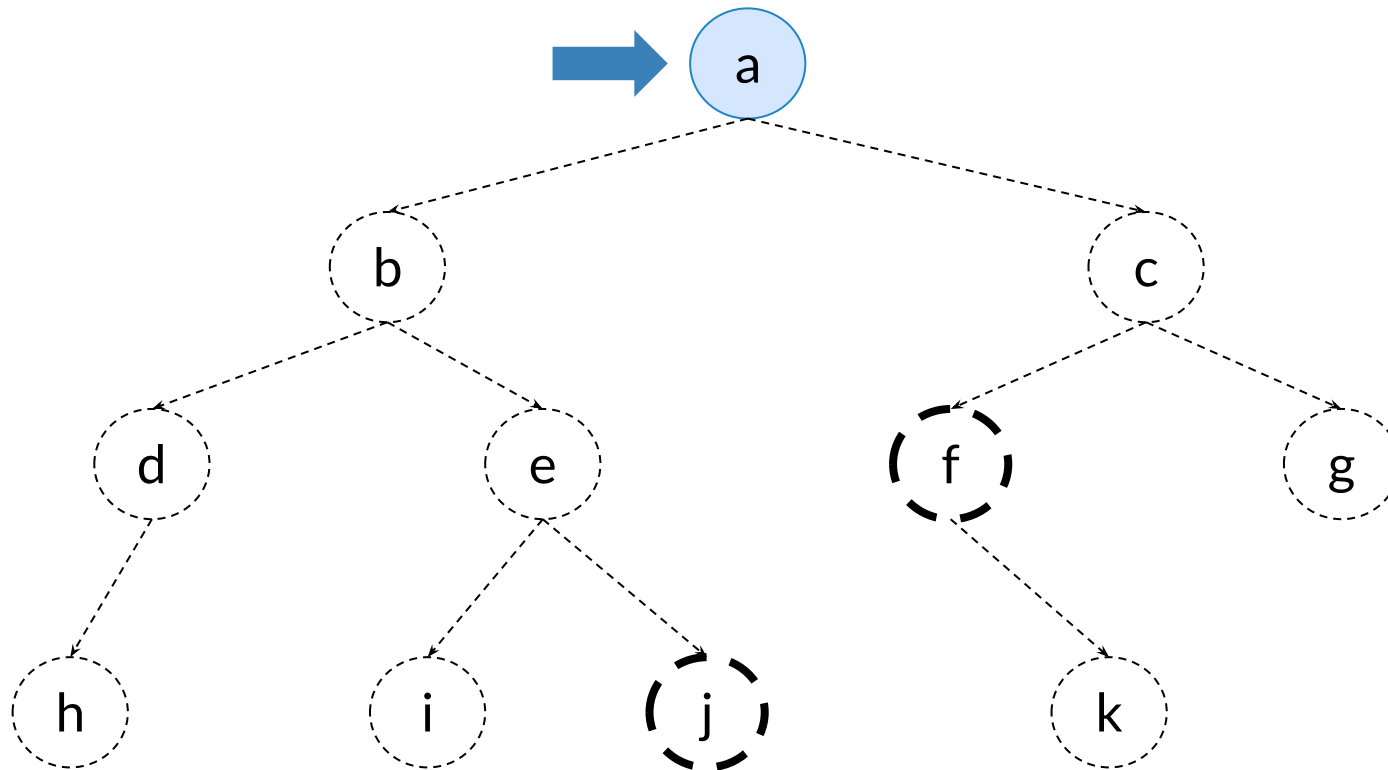
- ▷ Expande o nó raiz, depois todos os filhos da raiz, depois os filhos desses nós, e assim por diante
- ▷ Utiliza **fila (FIFO)** para expandir sempre o nó mais raso

Refinamento → teste de objetivo é aplicado a cada nó quando é gerado e não quando é selecionado para expansão

Busca em Largura

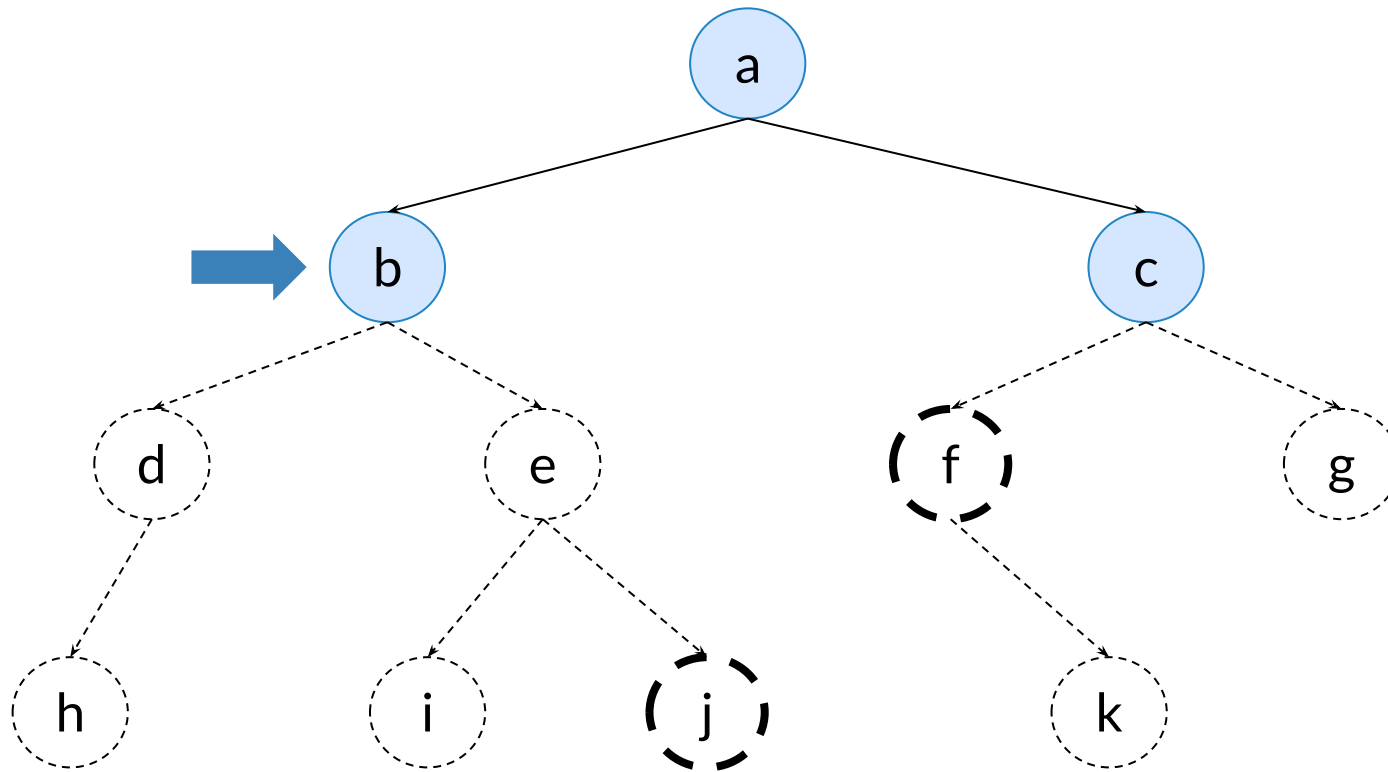
```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure  
  node  $\leftarrow$  NODE(problem.INITIAL)  
  if problem.IS-GOAL(node.STATE) then return node  
  frontier  $\leftarrow$  a FIFO queue, with node as an element  
  reached  $\leftarrow$  {problem.INITIAL}  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if problem.IS-GOAL(s) then return child  
      if s is not in reached then  
        add s to reached  
        add child to frontier  
  return failure
```

Busca em Largura



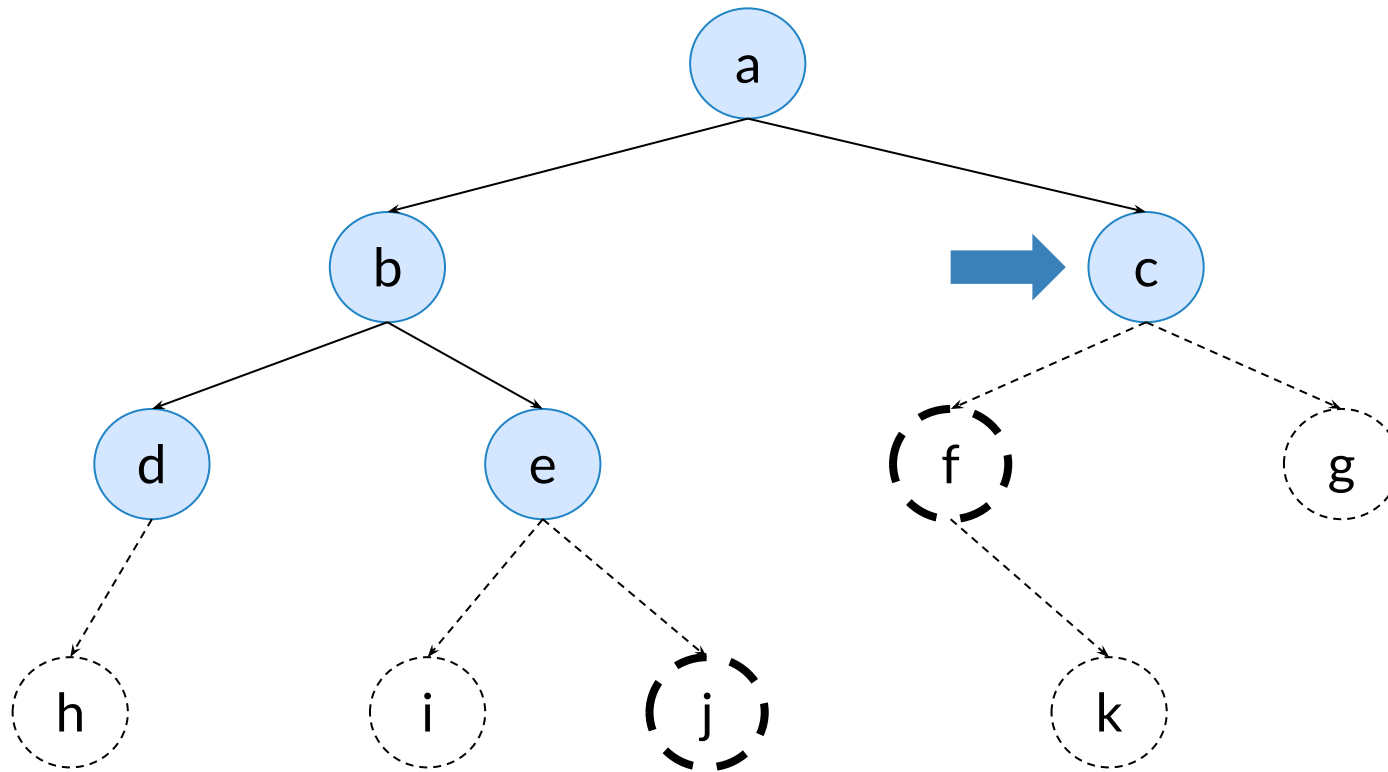
FIFO: a

Busca em Largura



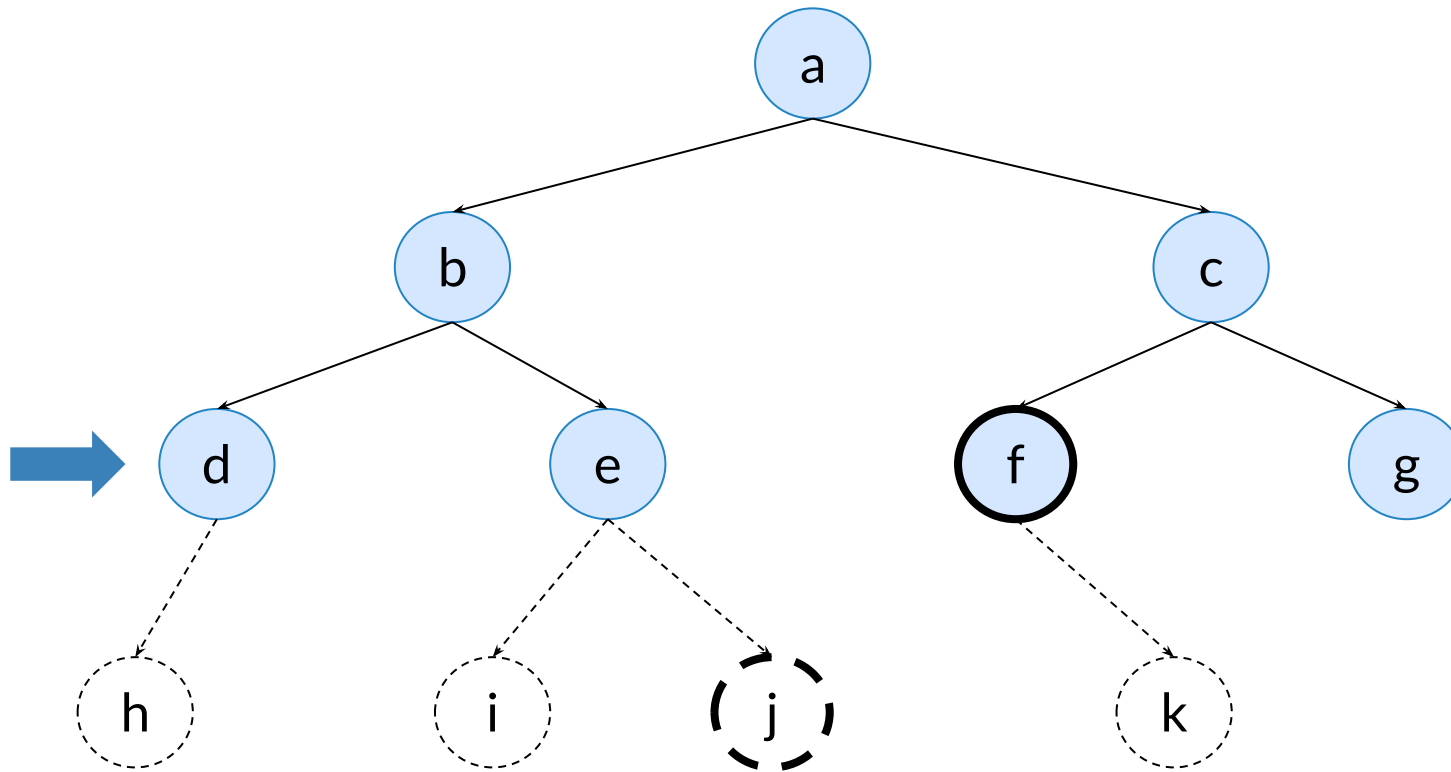
FIFO: b, c

Busca em Largura



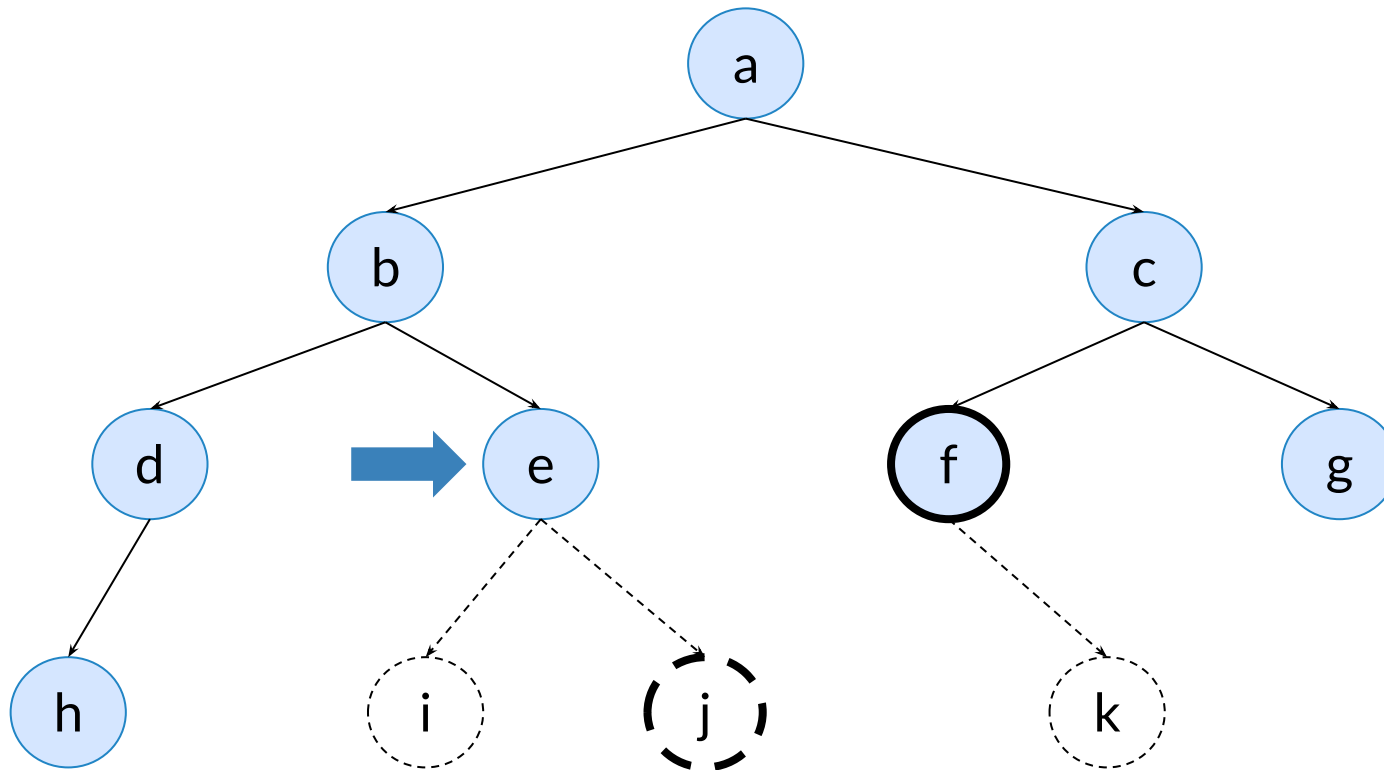
FIFO: c, d, e

Busca em Largura



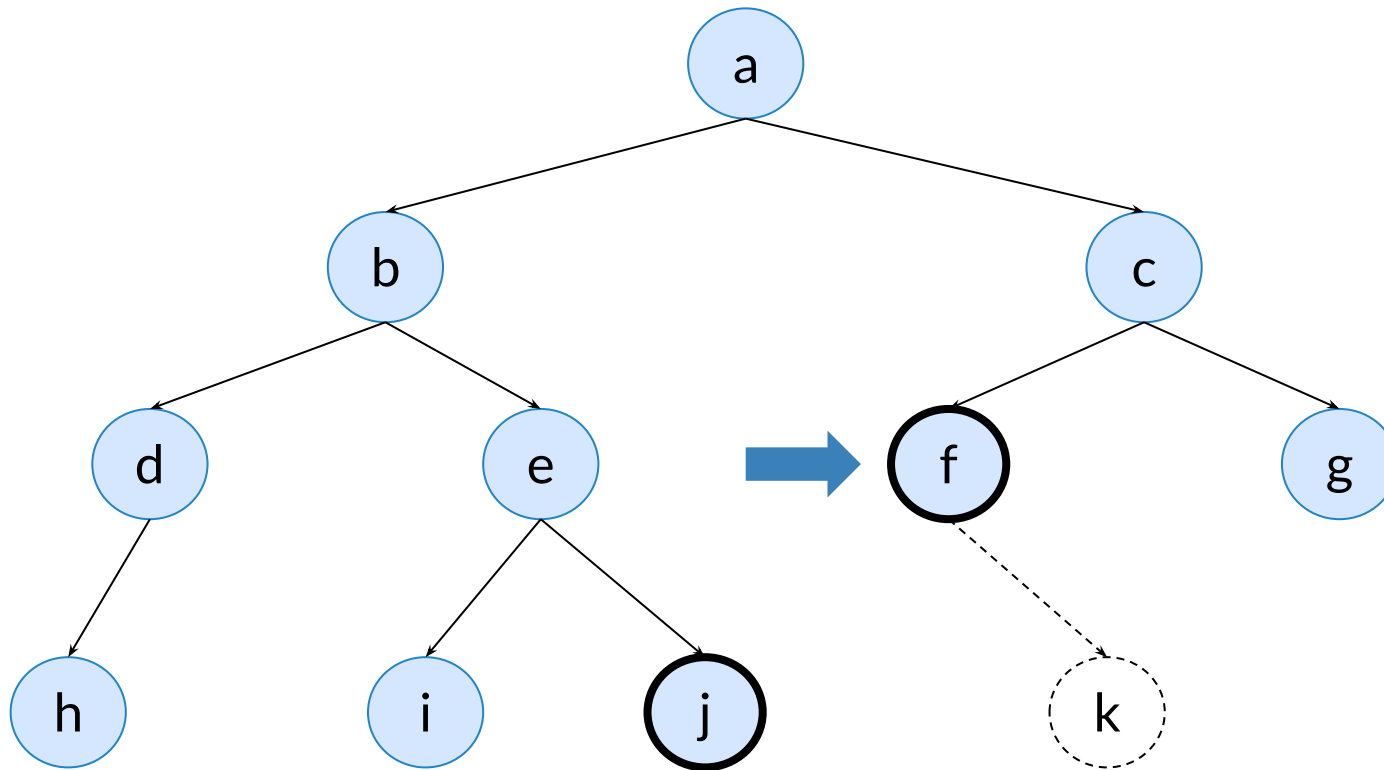
FIFO: d, e, f, g

Busca em Largura



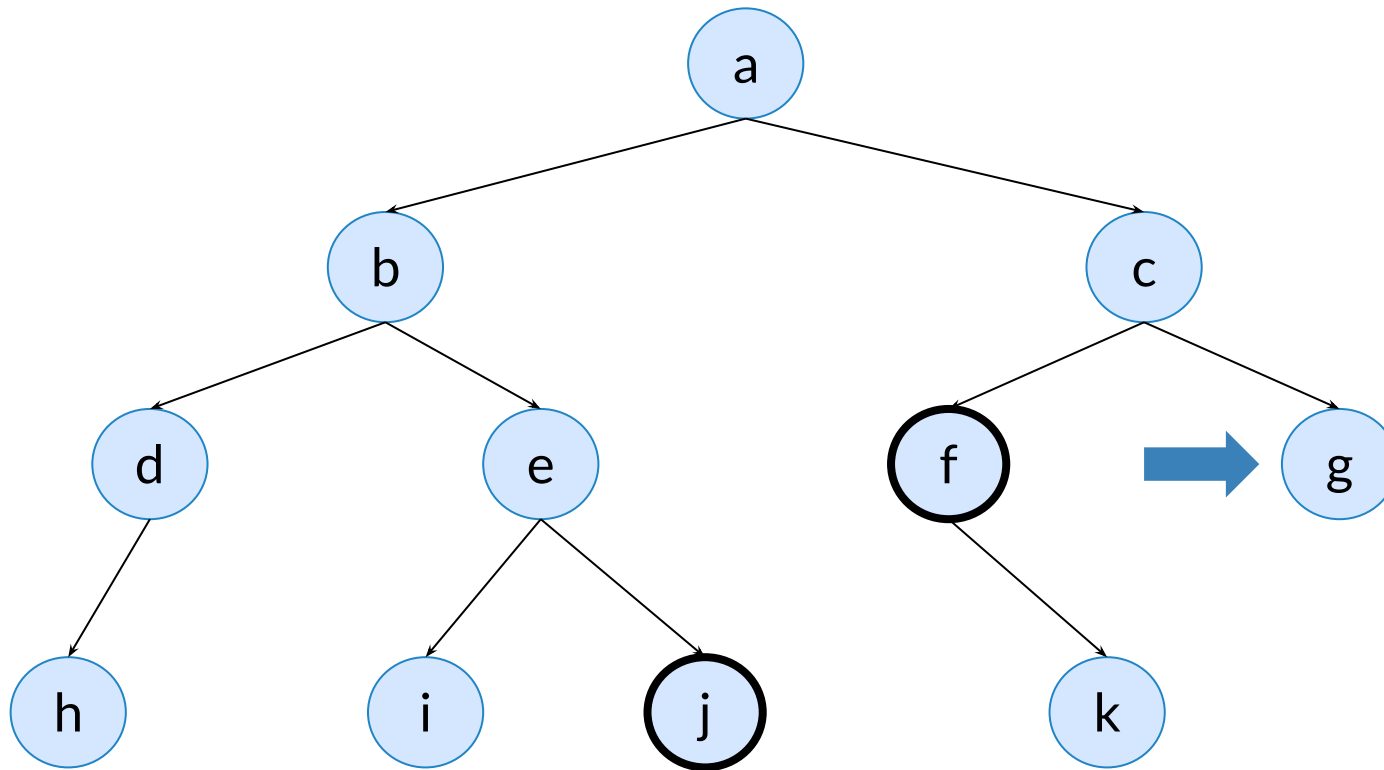
FIFO: e, f, g, h

Busca em Largura



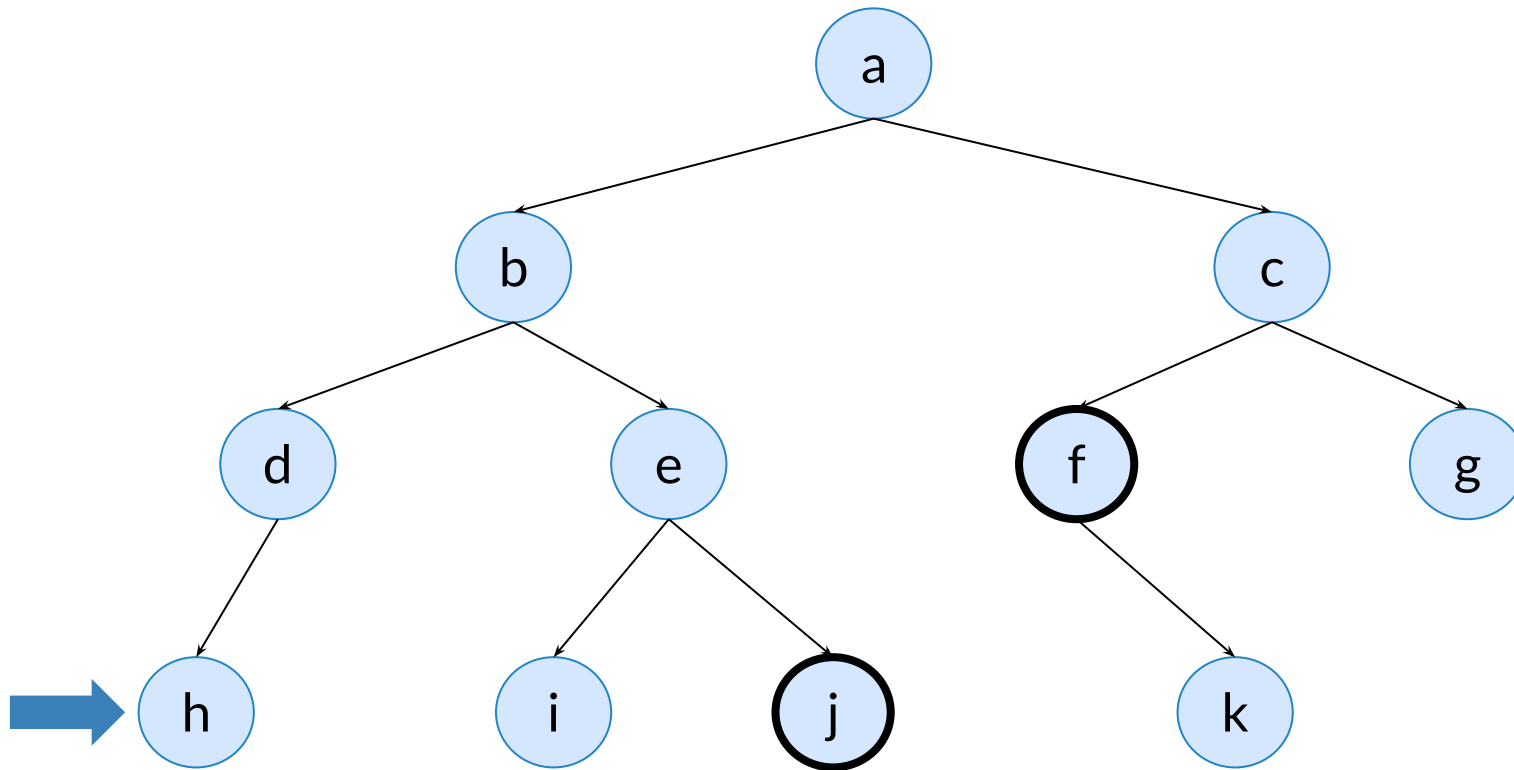
FIFO: f, g, h, i, j

Busca em Largura



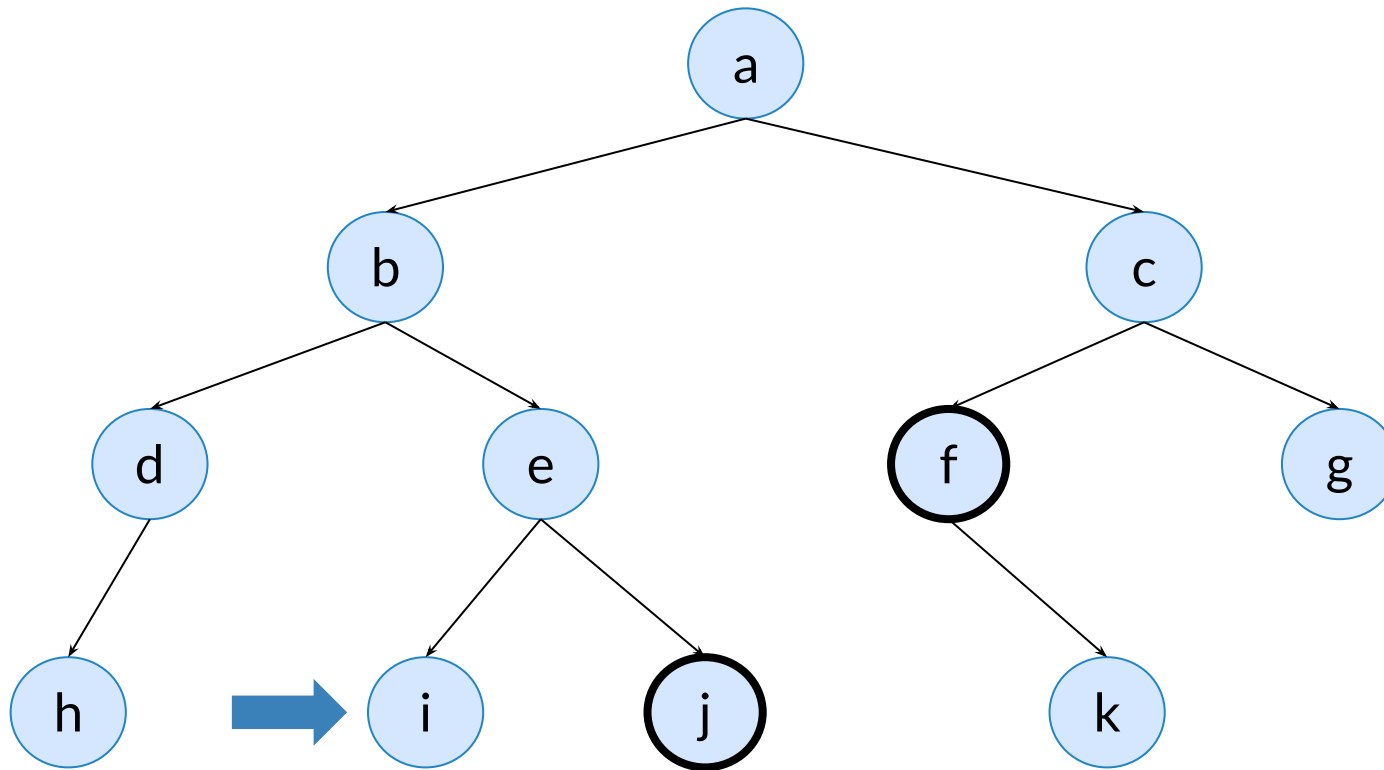
FIFO: g, h, i, j, k

Busca em Largura



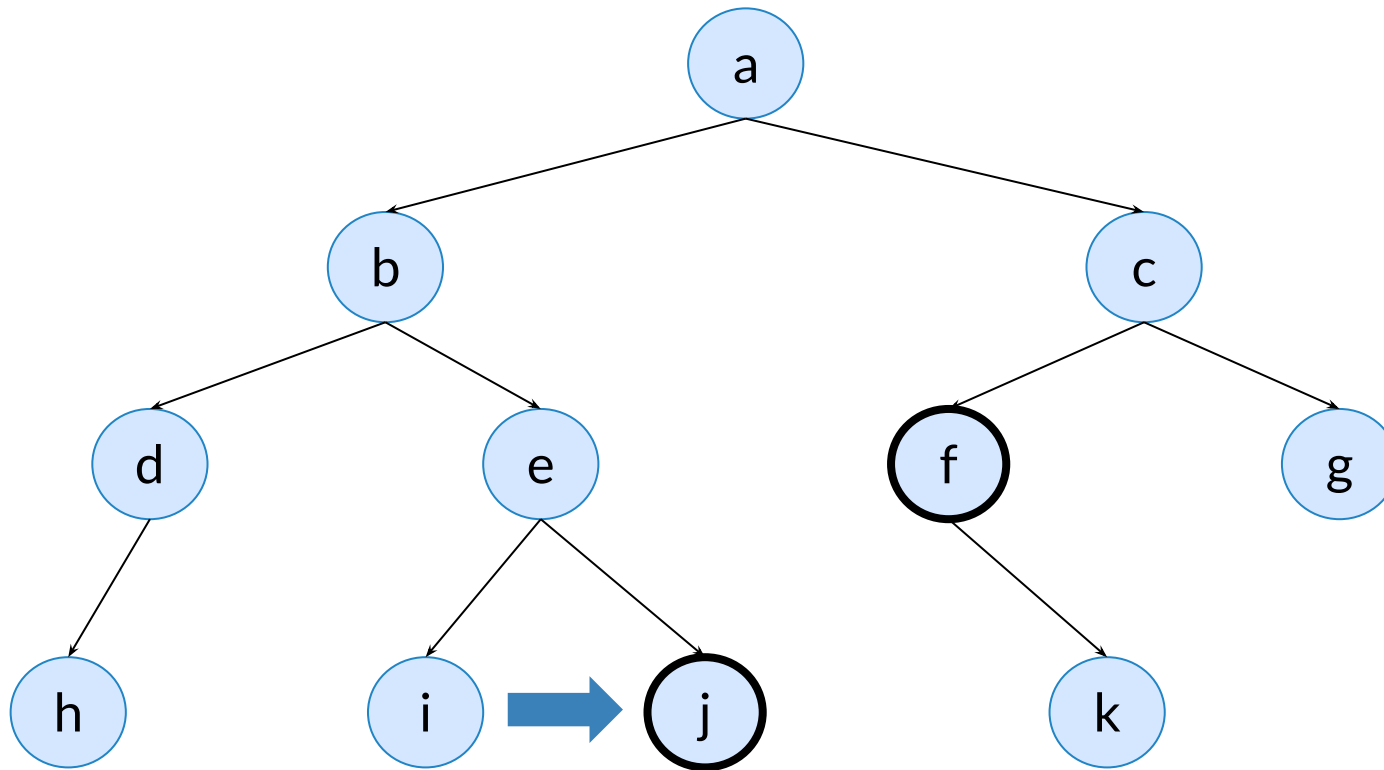
FIFO: h, i, j, k

Busca em Largura



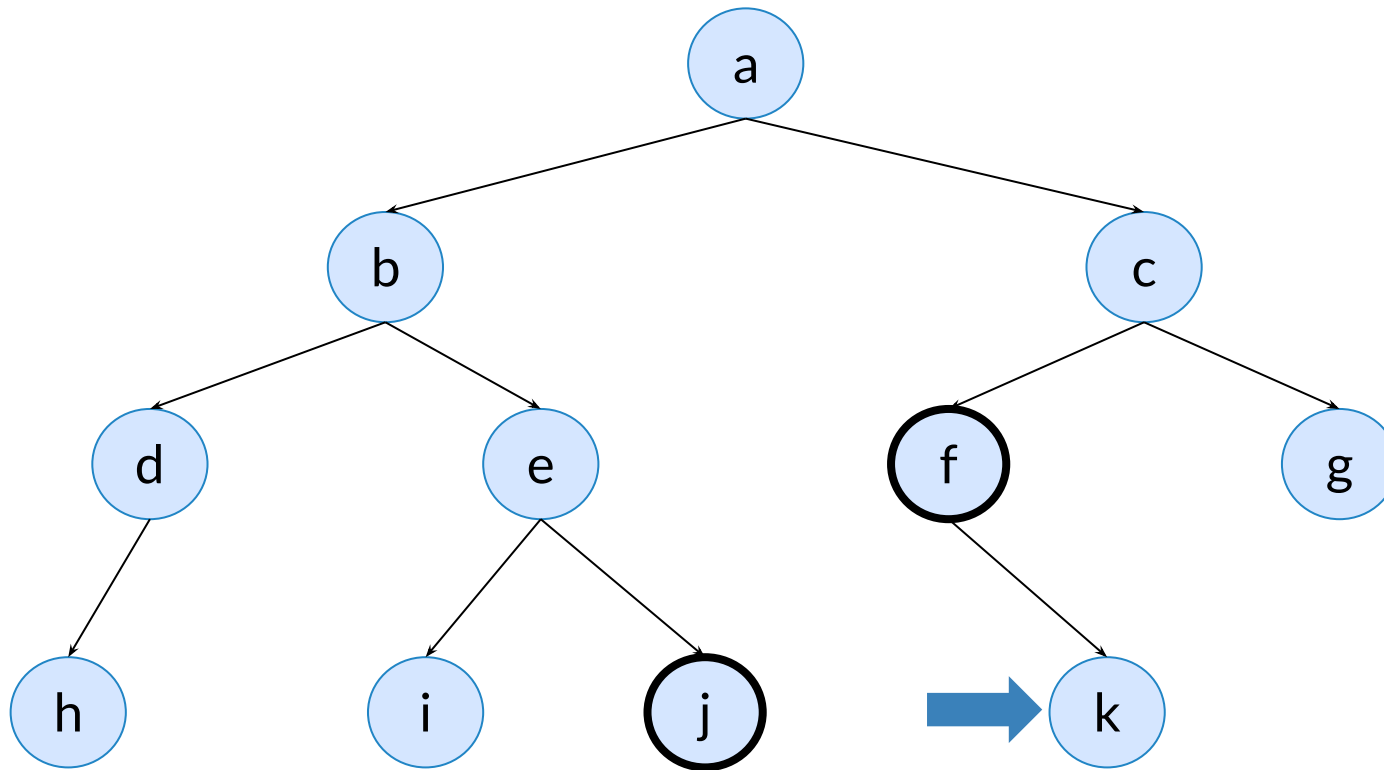
FIFO: i, j, k

Busca em Largura



FIFO: j, k

Busca em Largura



FIFO: k

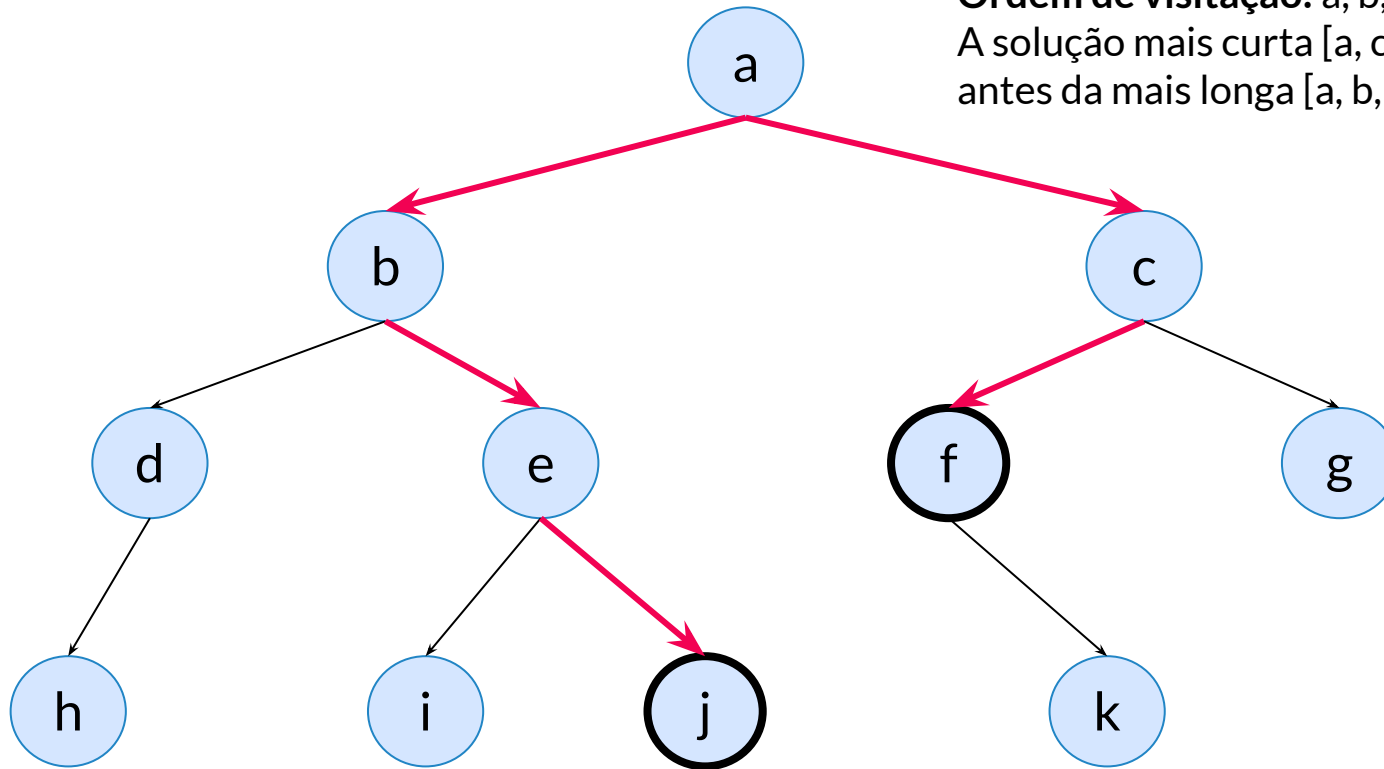
Busca em Largura

Estado inicial: a

Estados finais: f, j

Ordem de visitação: a, b, c, d, e, f

A solução mais curta [a, c, f] é encontrada antes da mais longa [a, b, e, j]



Busca em Largura – Resumo

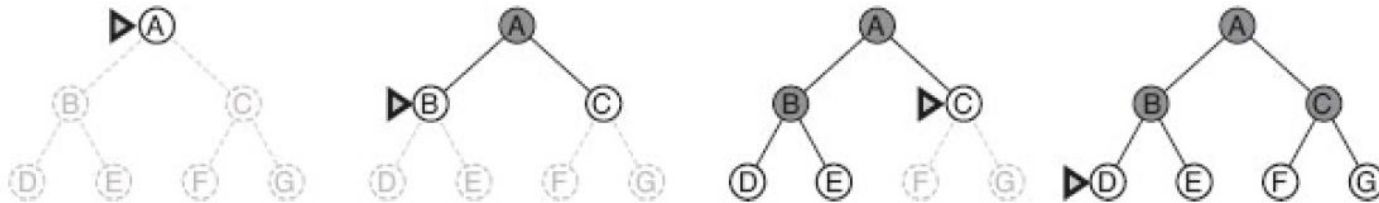


Figura 3.12 Busca em largura em uma árvore binária simples. Em cada fase, o próximo nó a ser expandido é indicado por um marcador.

Busca em Largura

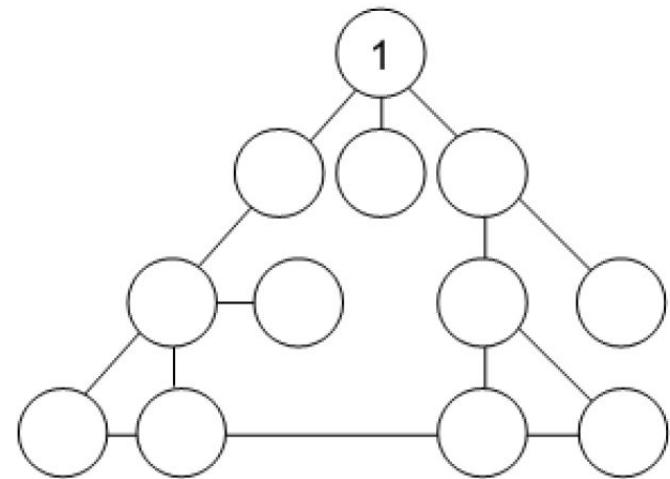
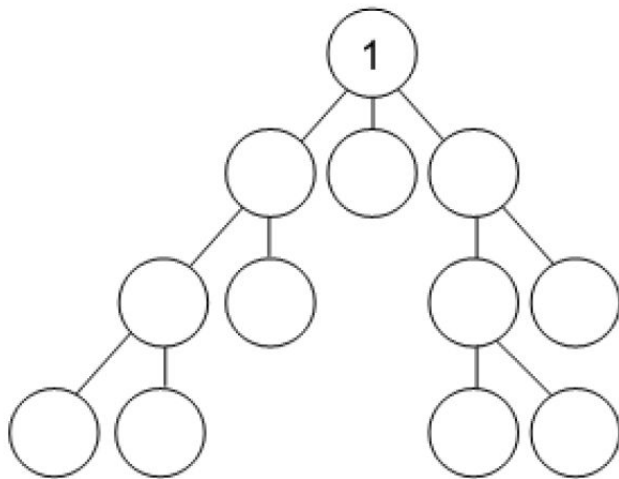
Avaliação do algoritmo

- ▷ **Completo:** sim (se a profundidade d for finita)
- ▷ **Ótimo:** sim, desde que o custo seja uma função não decrescente da profundidade do nó
- ▷ **Complexidade**
 - Considerando branch factor b , solução no nível d
 - Cada estado tem b sucessores
 - Raiz gera b nós no primeiro nível, cada um desses outros gera outros b outros nós, totalizando b^2 no segundo nível, e assim sucessivamente
 - Tempo: $b + b^2 + b^3 + \dots + b^d = O(b^d)$
 - Espaço: similar, pois todos os nós são mantidos em memória

Exercício #1

Indique a ordem na qual os nós são visitados na busca em largura

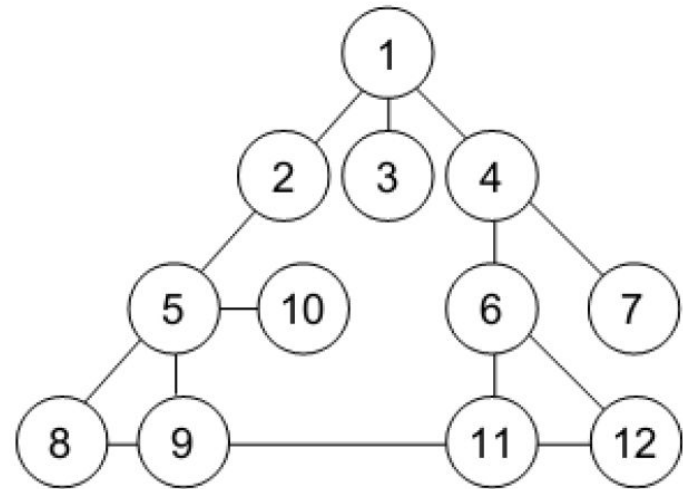
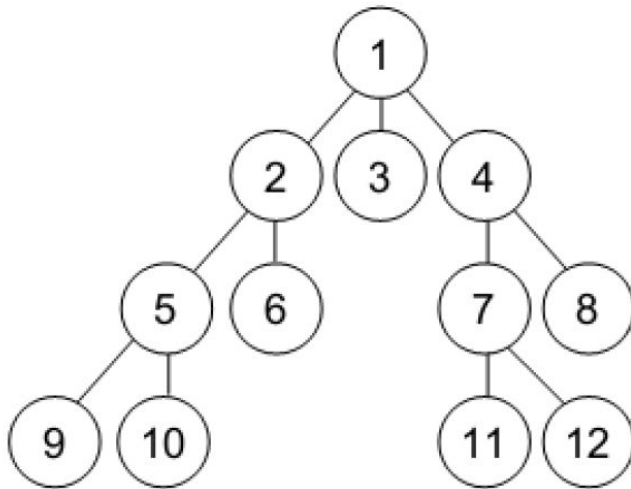
- Assuma que os sucessores são definidos da esquerda para a direita, e, depois, de cima para baixo



Exercício #1 – Solução

Indique a ordem na qual os nós são visitados na busca em largura

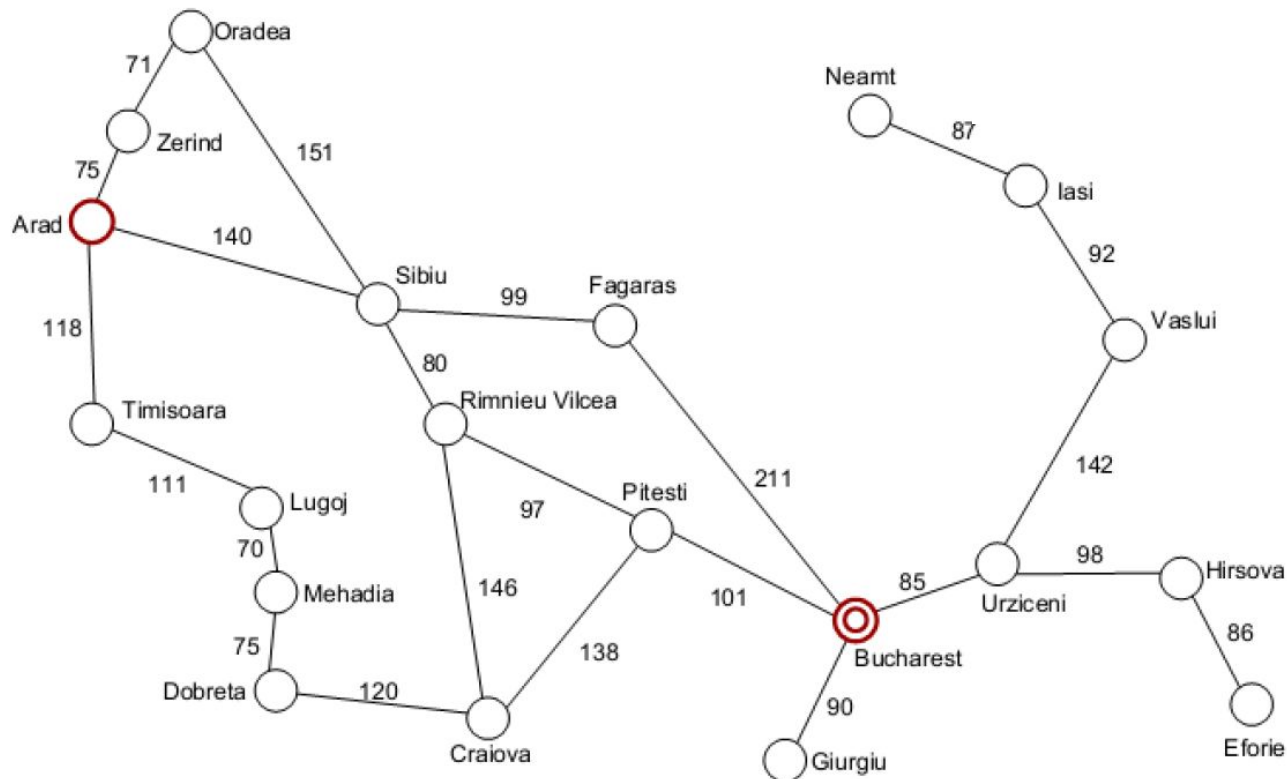
- Assuma que os sucessores são definidos da esquerda para a direita, e, depois, de cima para baixo



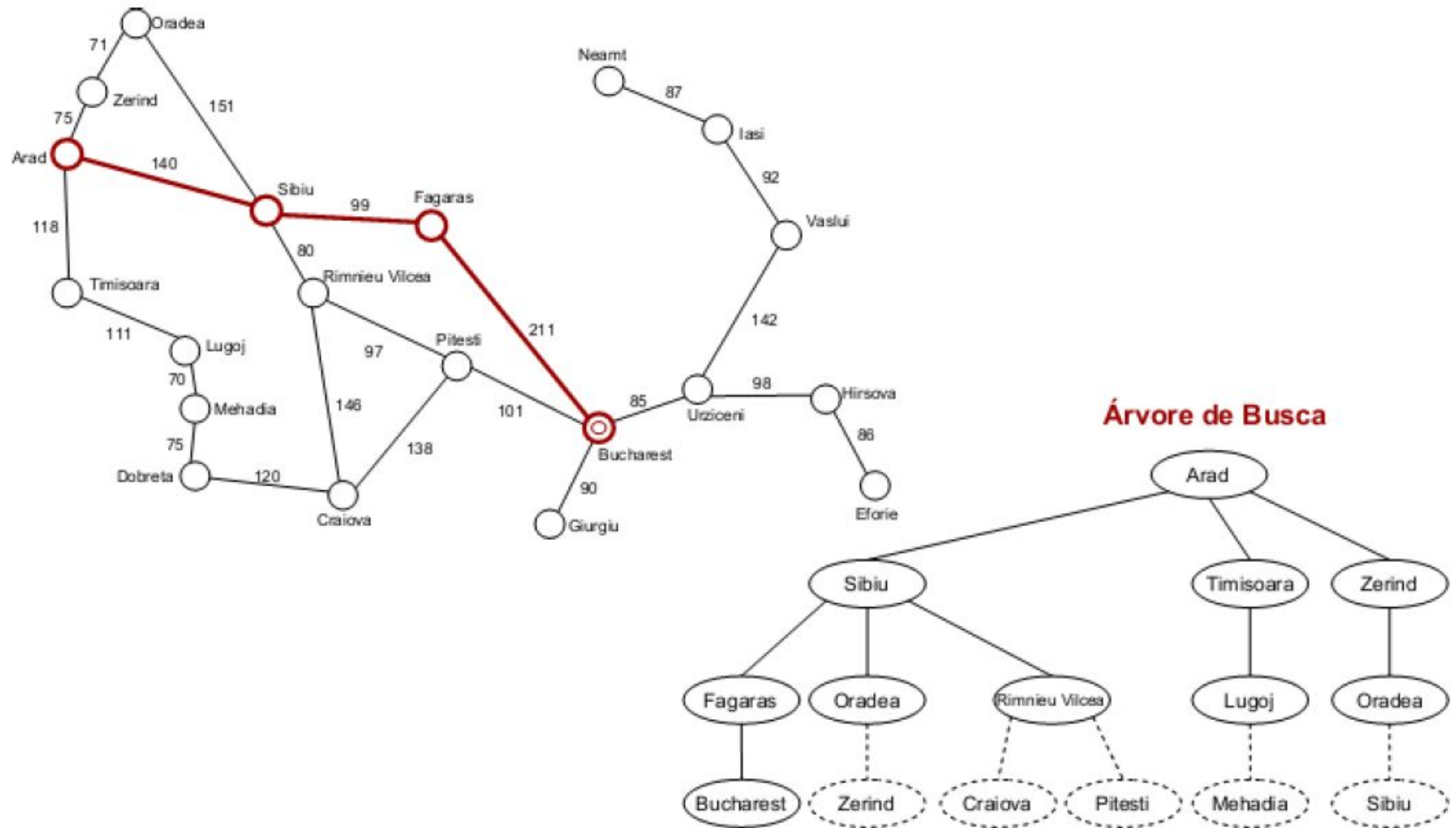
Exercício #2

Encontre o caminho e a árvore de busca de Arad até Bucareste usando a Busca em Largura

- Assuma que os sucessores de um nó são definidos em ordem alfabética



Exercício #2 – Solução



4.

Busca de Custo Uniforme

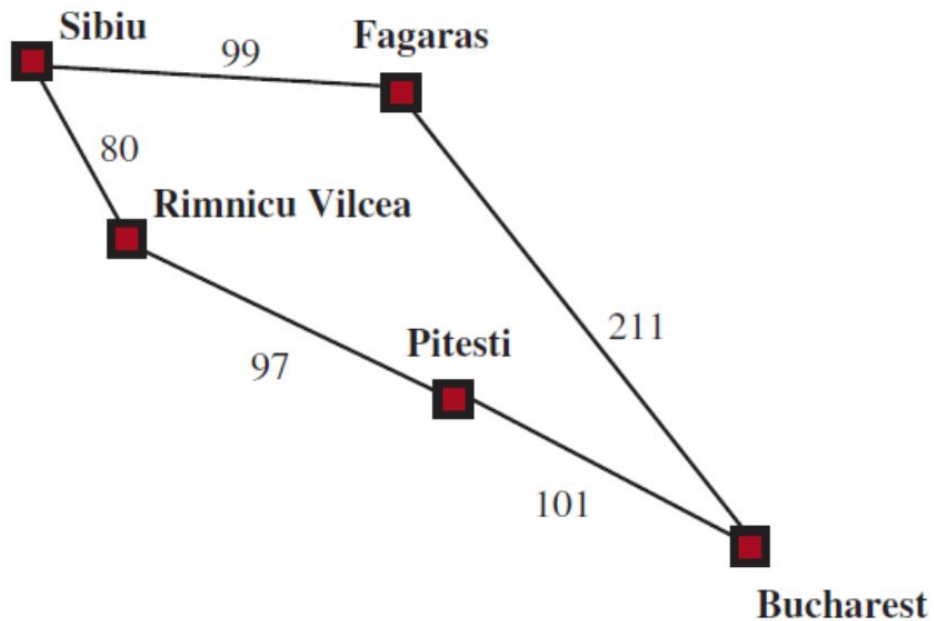
Também conhecida como Algoritmo de Dijkstra

Busca de Custo Uniforme

- ▷ Similar à busca em largura (BFS), exceto pelo fato que os caminhos são colocados em uma **fila de prioridades** (heap)
 - Se comporta como uma fila, mas os elementos são sempre ordenados em relação a algum valor
- ▷ A busca de custo uniforme **expande o nó com o caminho de menor custo** até o momento
 - Se os custos forem todos iguais, se torna BFS

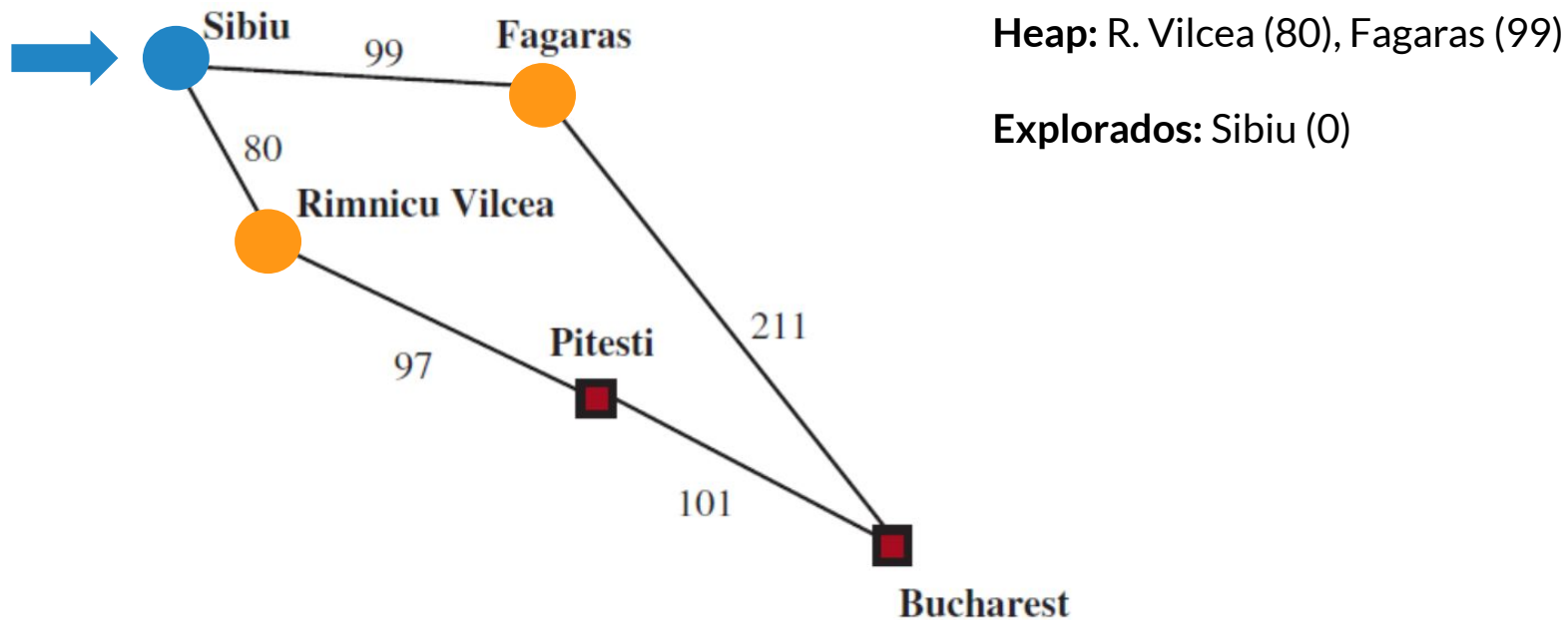
Busca de Custo Uniforme

Considere o problema de viajar de Sibiu a Bucareste



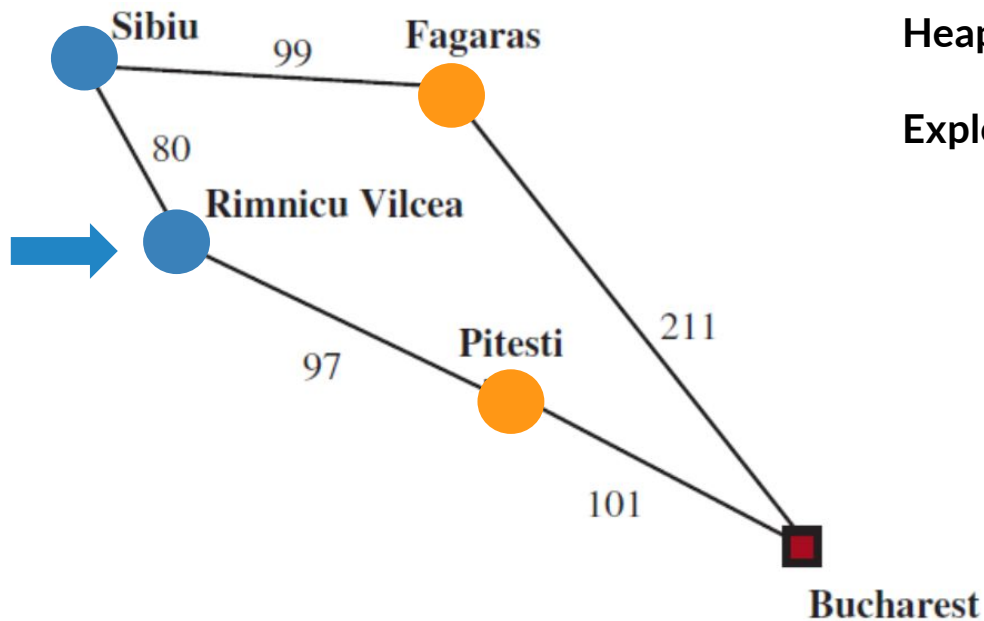
Busca de Custo Uniforme

Considere o problema de viajar de Sibiu a Bucareste



Busca de Custo Uniforme

Considere o problema de viajar de Sibiu a Bucareste

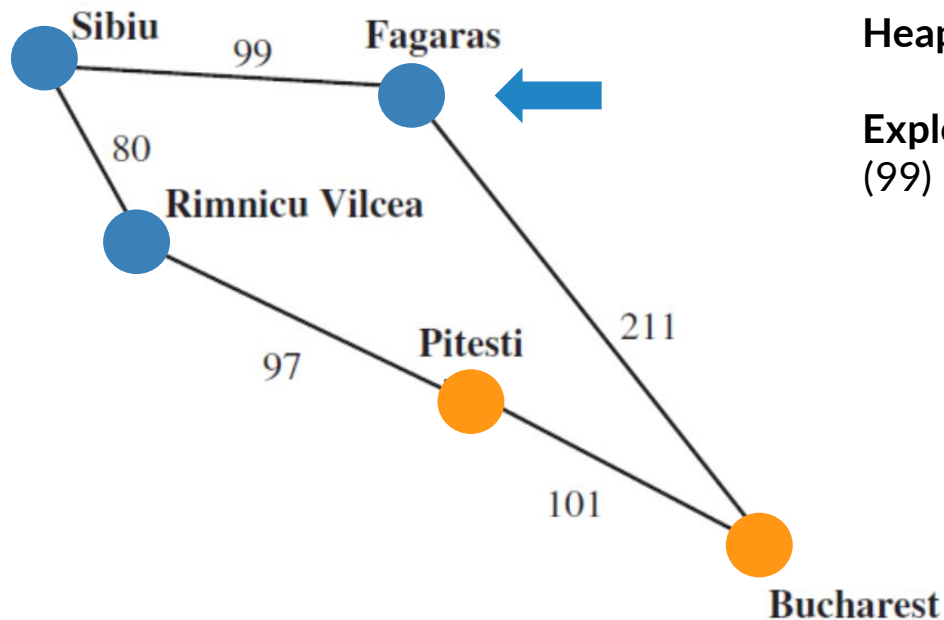


Heap: Fagaras (99), Pitesti (177)

Explorados: Sibiu (0), R. Vilcea (80)

Busca de Custo Uniforme

Considere o problema de viajar de Sibiu a Bucareste

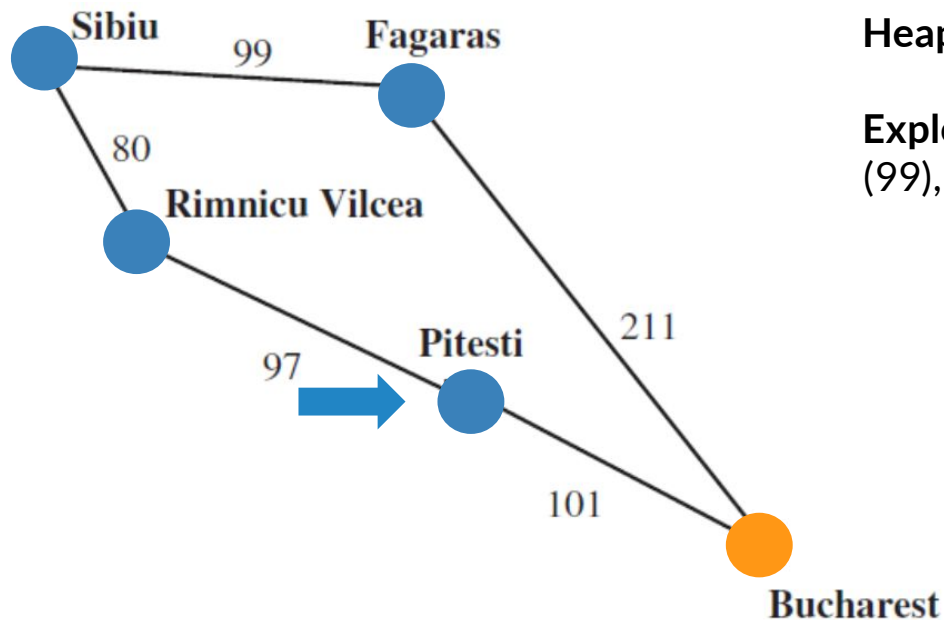


Heap: Pitesti (177), Bucharest (310)

Explorados: Sibiu (0), R. Vilcea (80), Fagaras (99)

Busca de Custo Uniforme

Considere o problema de viajar de Sibiu a Bucareste

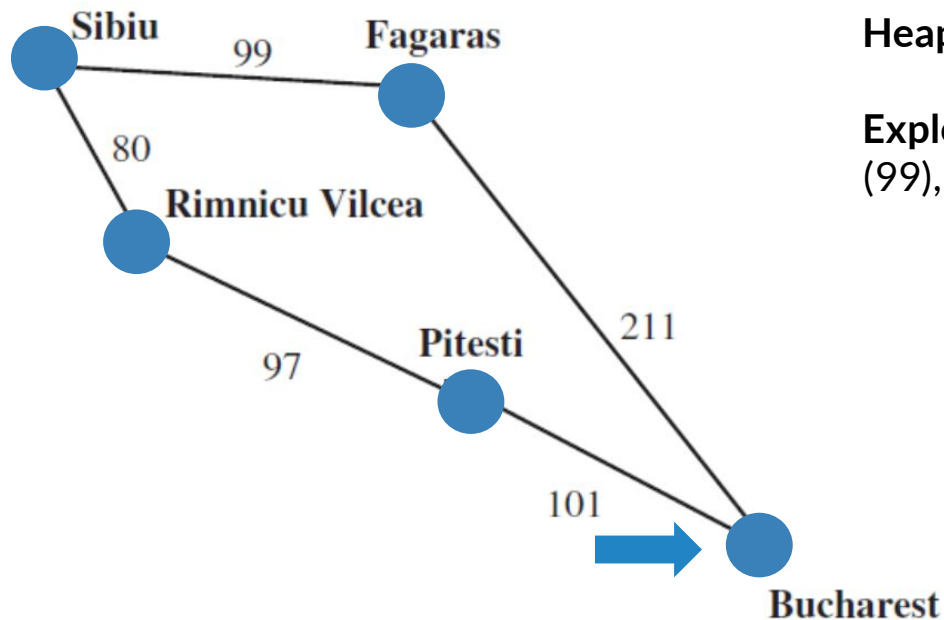


Heap: Bucharest (~~310~~ 278)

Explorados: Sibiu (0), R. Vilcea (80), Fagaras (99), Pitesti (177)

Busca de Custo Uniforme

Considere o problema de viajar de Sibiu a Bucareste



Heap:

Explorados: Sibiu (0), R. Vilcea (80), Fagaras (99), Pitesti (177), Bucharest (278)

Busca de Custo Uniforme

Avaliação do algoritmo

- ▷ **Completo:** sim (se cada passo tem custo $\geq \epsilon$)
- ▷ **Ótimo:** sim (sempre vai pelo menor custo)
- ▷ **Complexidade**
 - Considerando C^* o custo da solução ótima
 - Tempo: $O(b^{1 + \lfloor C^* / \epsilon \rfloor})$
 - Espaço: similar

5.

Busca em Profundidade

Depth-first search

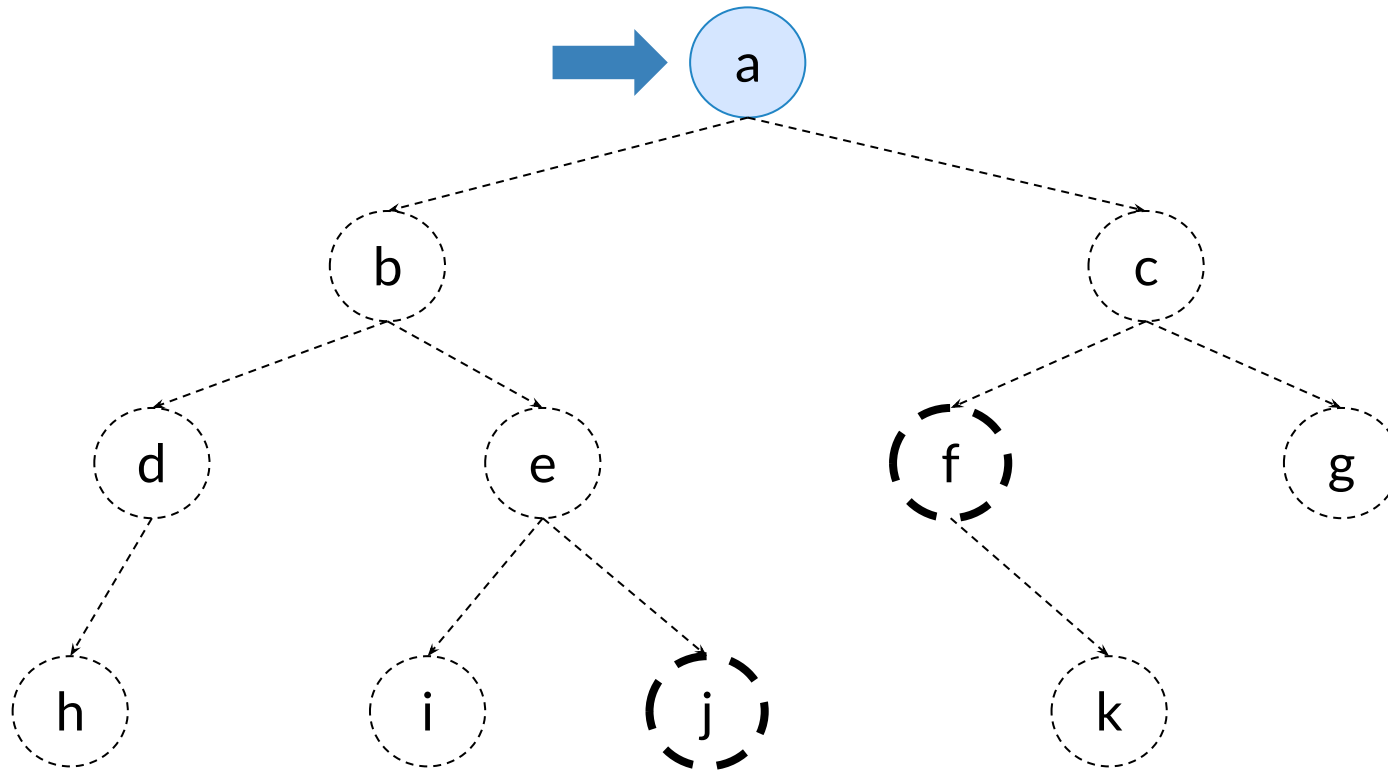
Busca em Profundidade

- ▷ Sempre expande o nó **mais profundo** na borda atual da árvore de busca
- ▷ A busca prossegue até o nível mais profundo da árvore de busca, onde os nós não têm sucessores
- ▷ Utiliza **pilha (LIFO)** para expandir sempre o nó gerado mais recentemente (profundo)

Busca em Profundidade

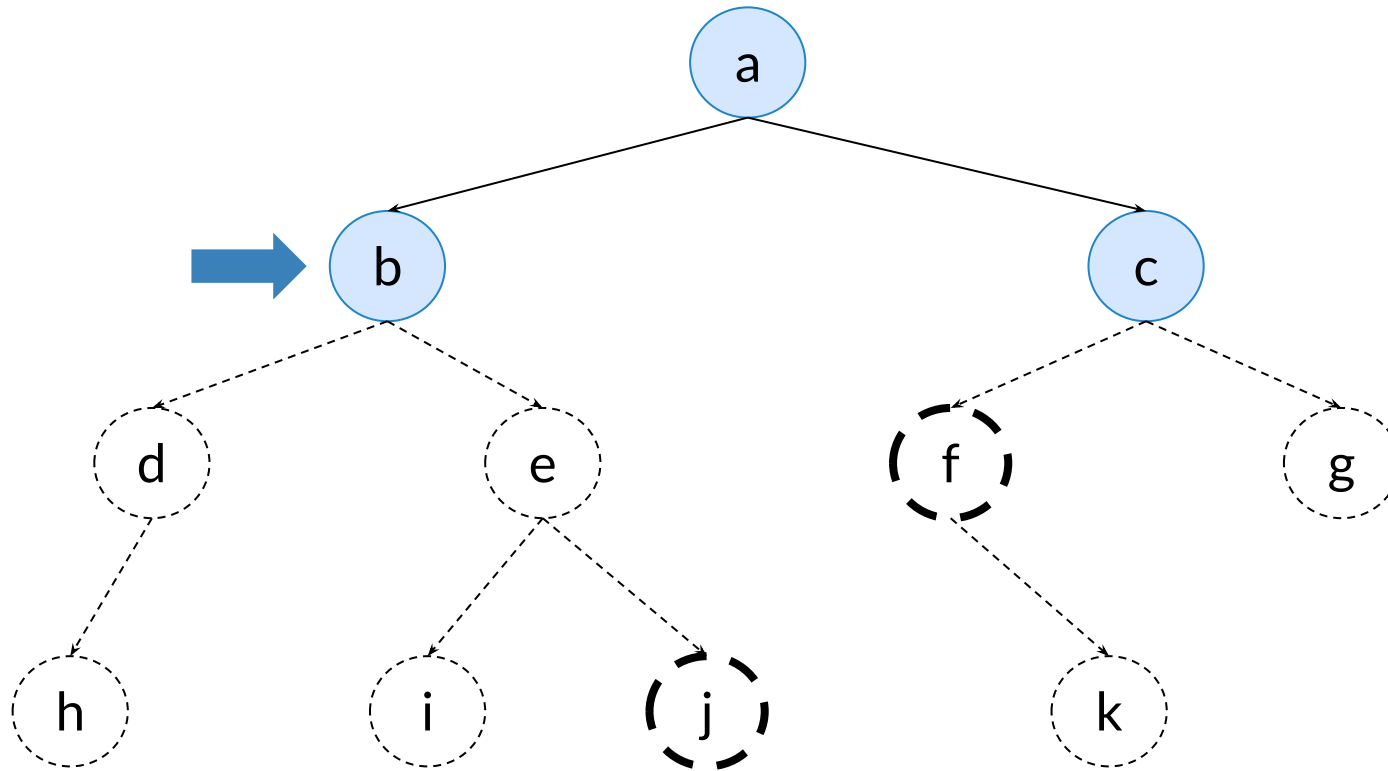
- ▷ Quando todas arestas adjacentes a um nó tiverem sido exploradas, a busca “volta para trás” (do inglês *backtracking*) para explorar vértices do qual esse nó foi descoberto
- ▷ O processo continua até que todos os nós tenham sido descobertos

Busca em Profundidade



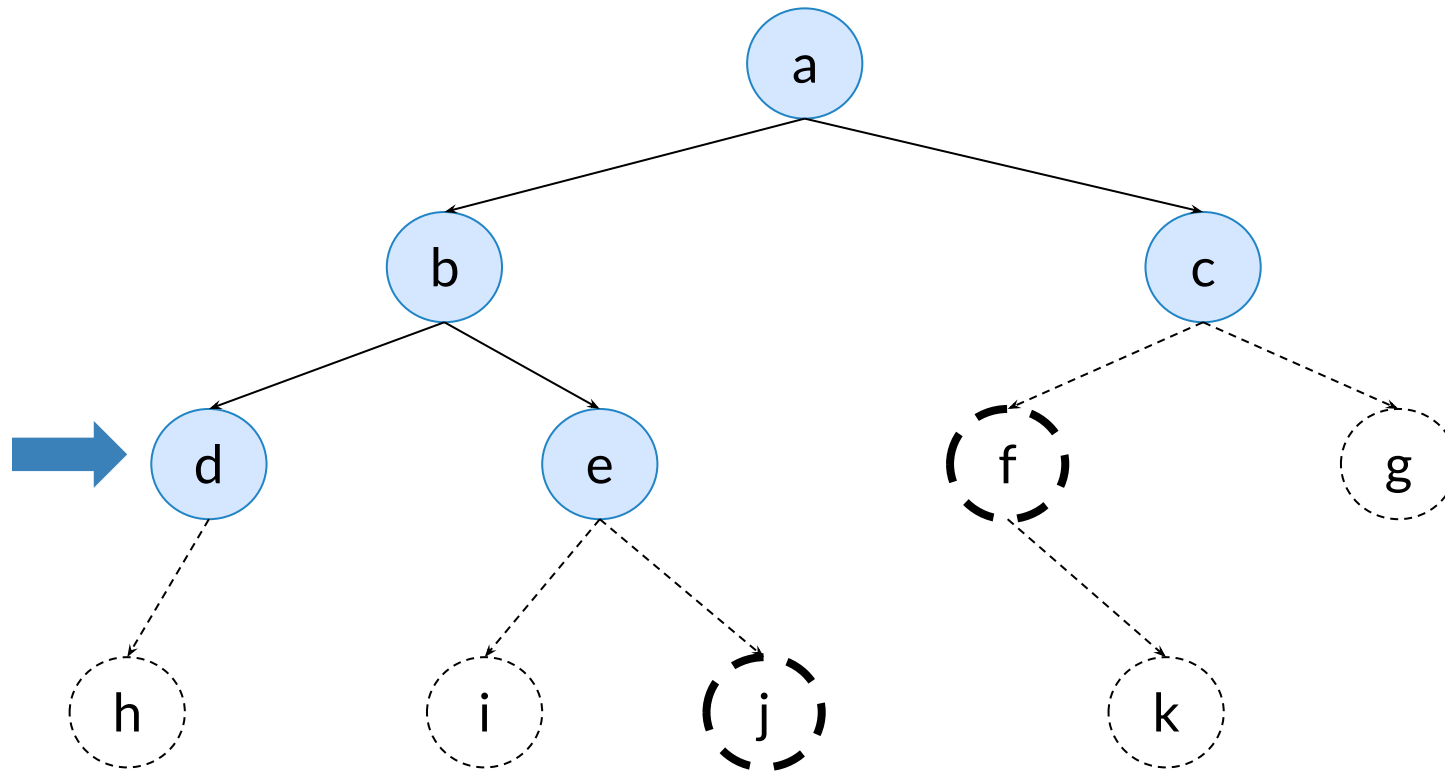
LIFO: a

Busca em Profundidade



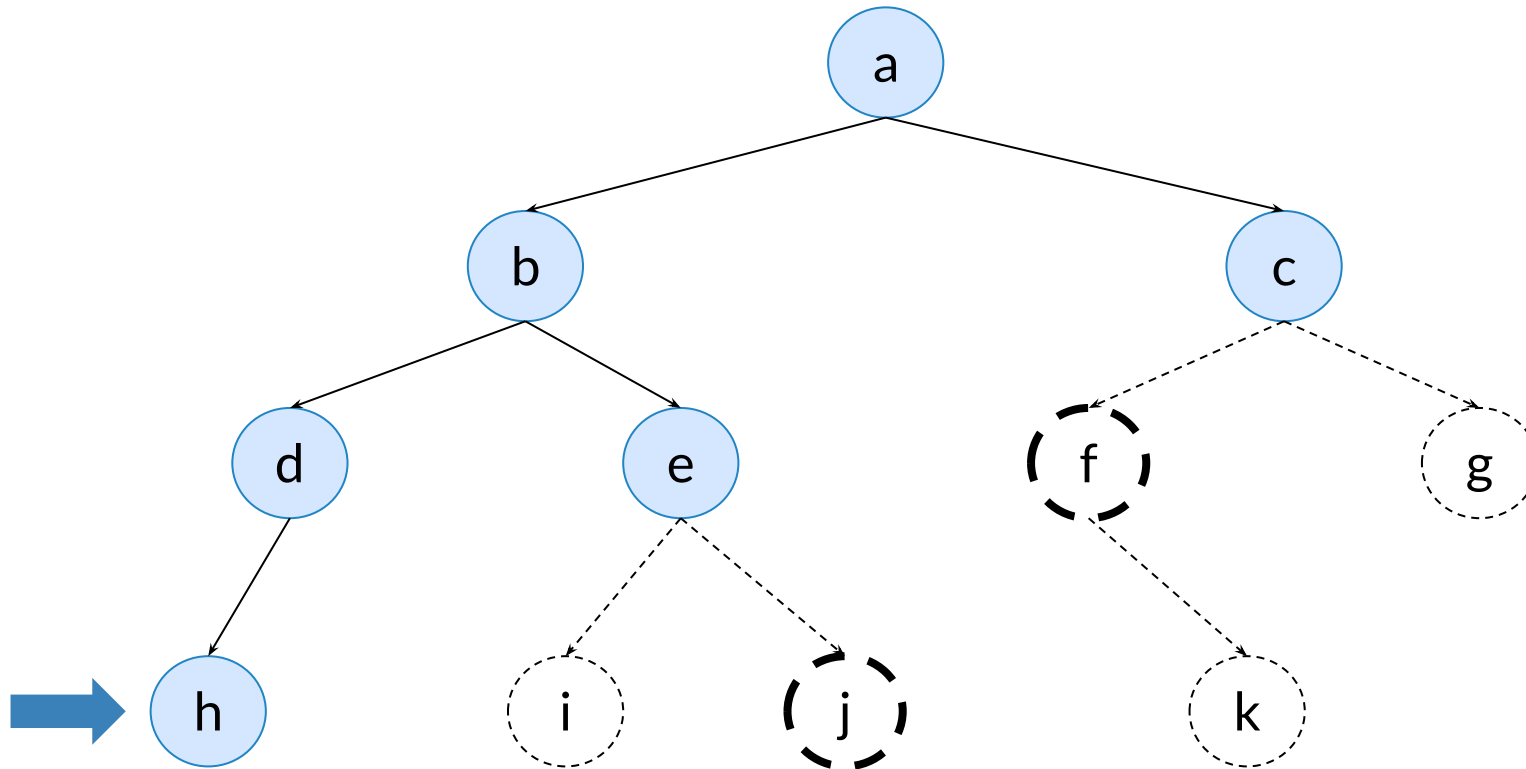
LIFO: b, c

Busca em Profundidade



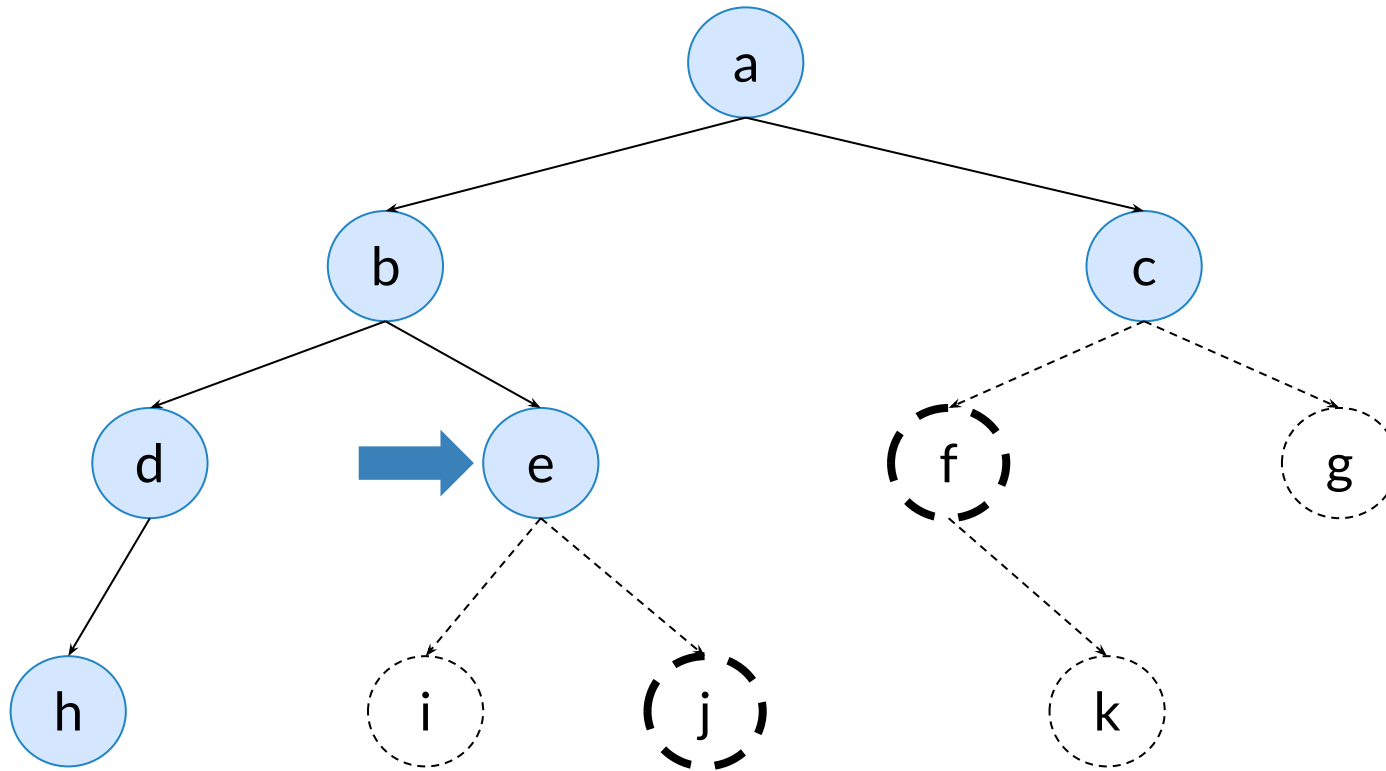
LIFO: d, e, c

Busca em Profundidade



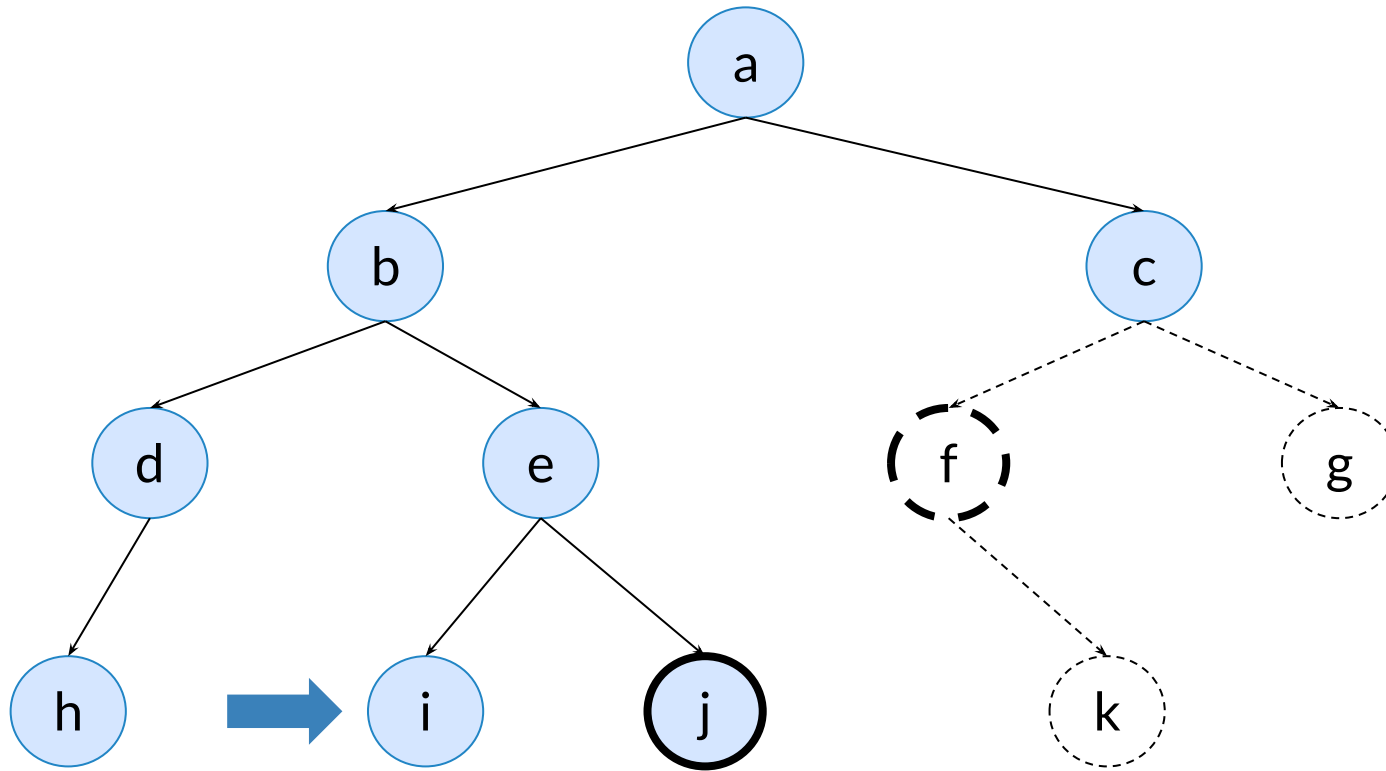
LIFO: h, e, c

Busca em Profundidade



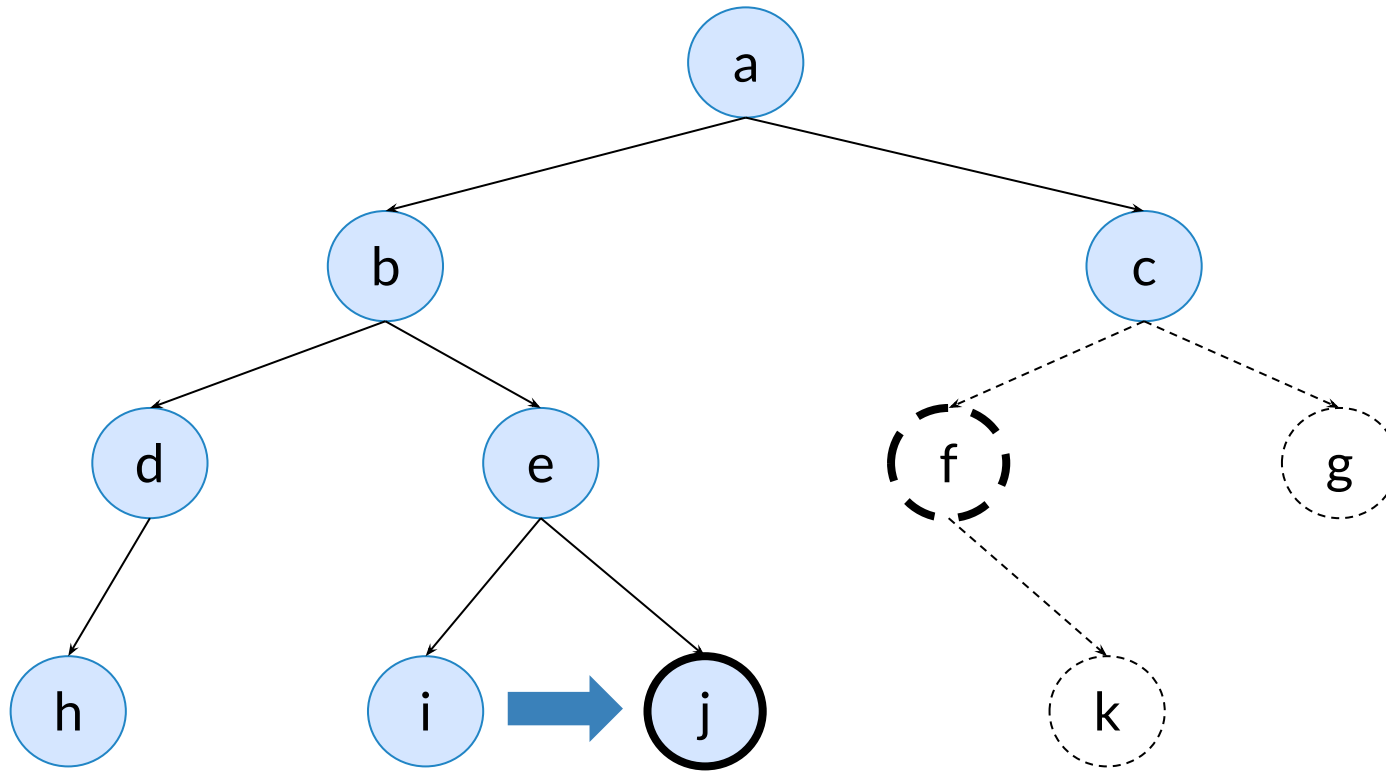
LIFO: e, c

Busca em Profundidade



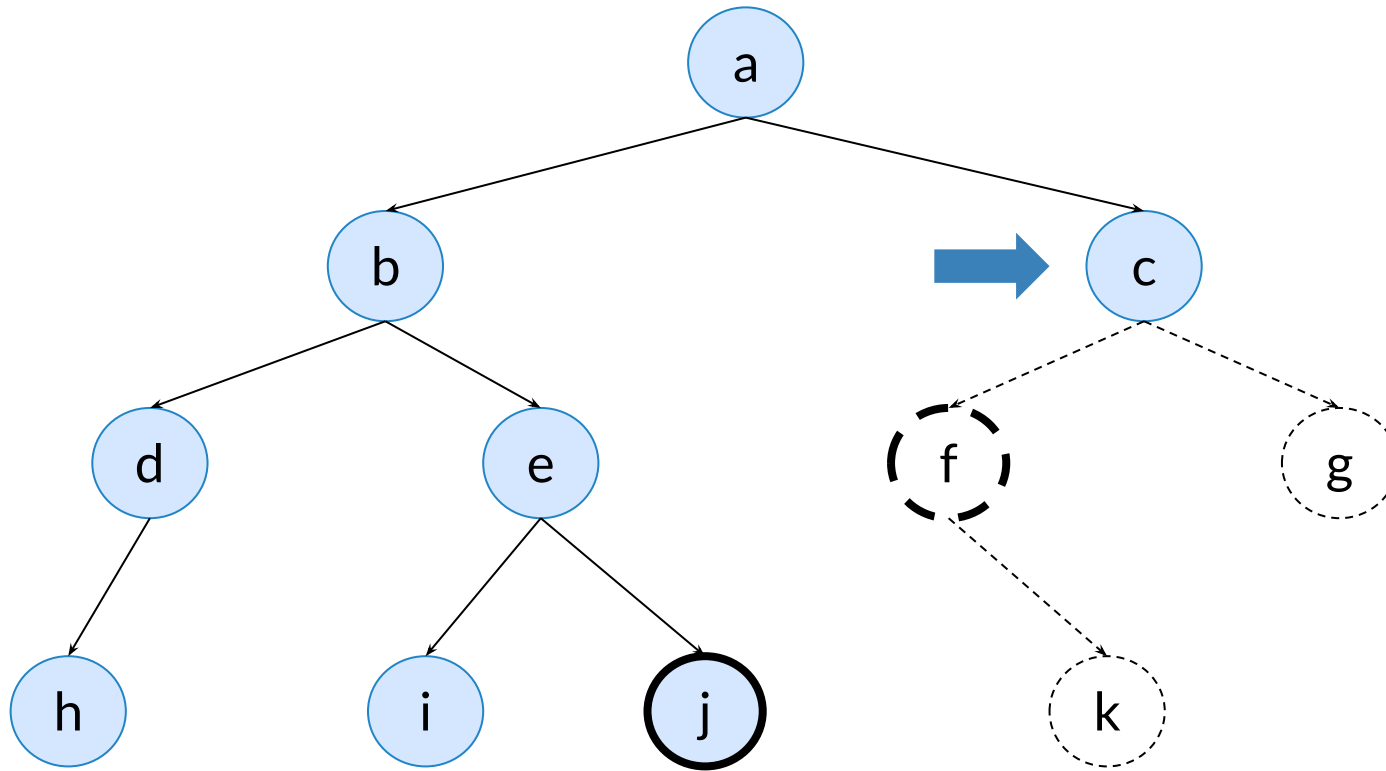
LIFO: i, j, c

Busca em Profundidade



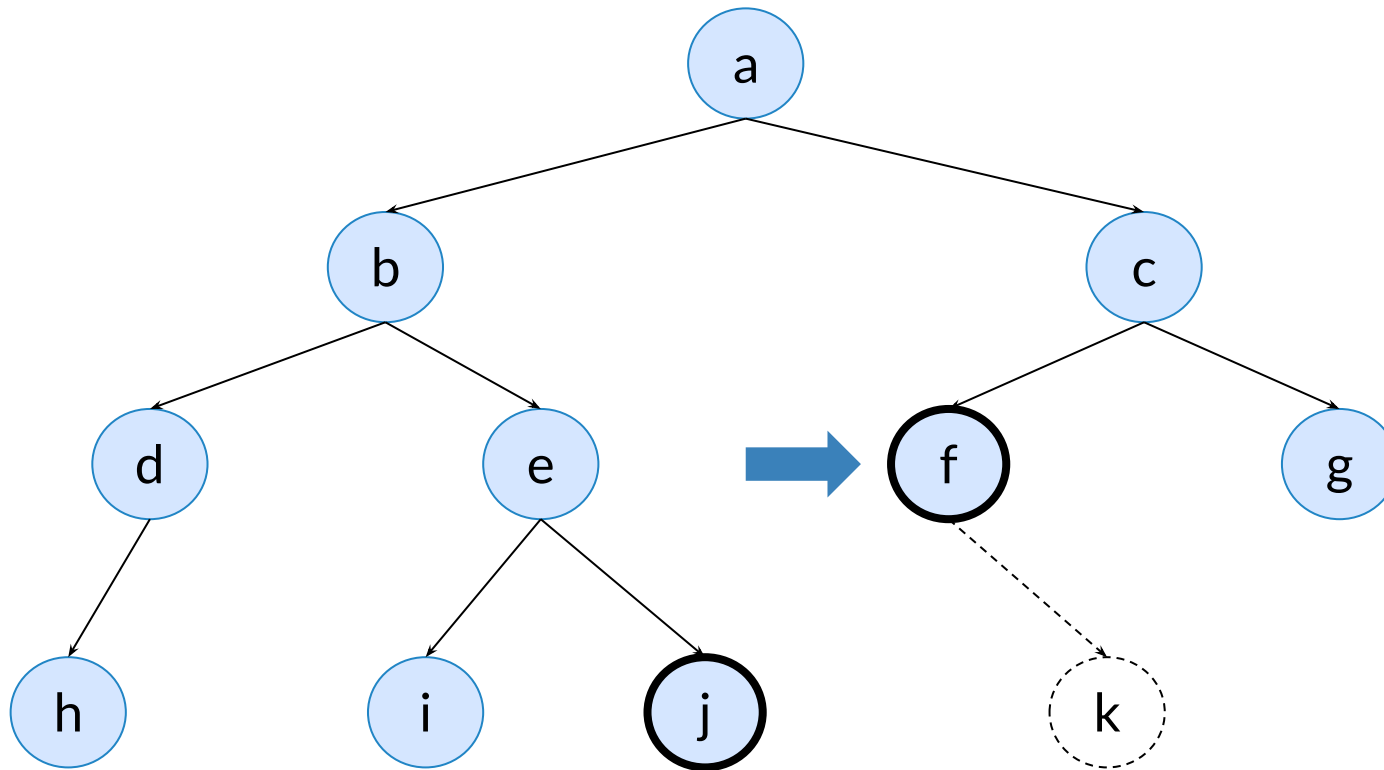
LIFO: j, c

Busca em Profundidade



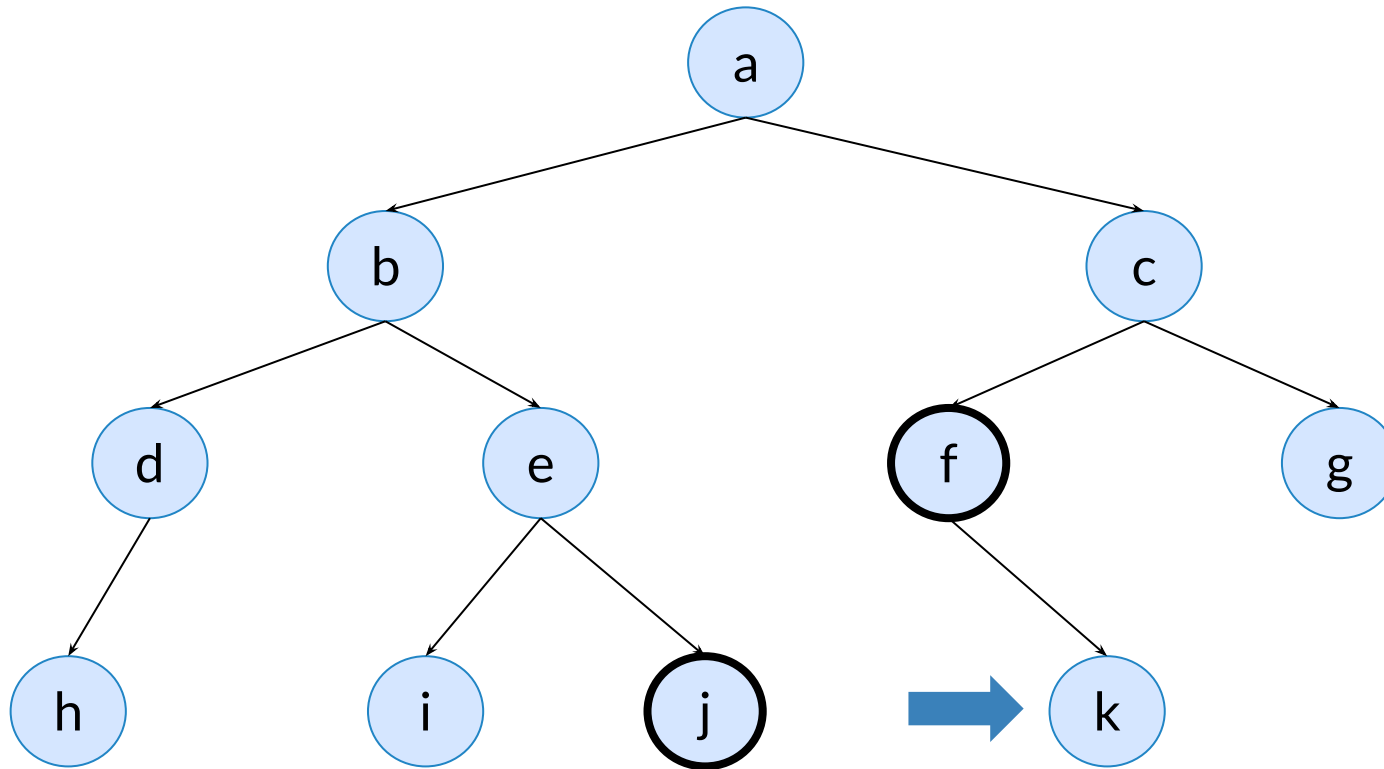
LIFO: c

Busca em Profundidade



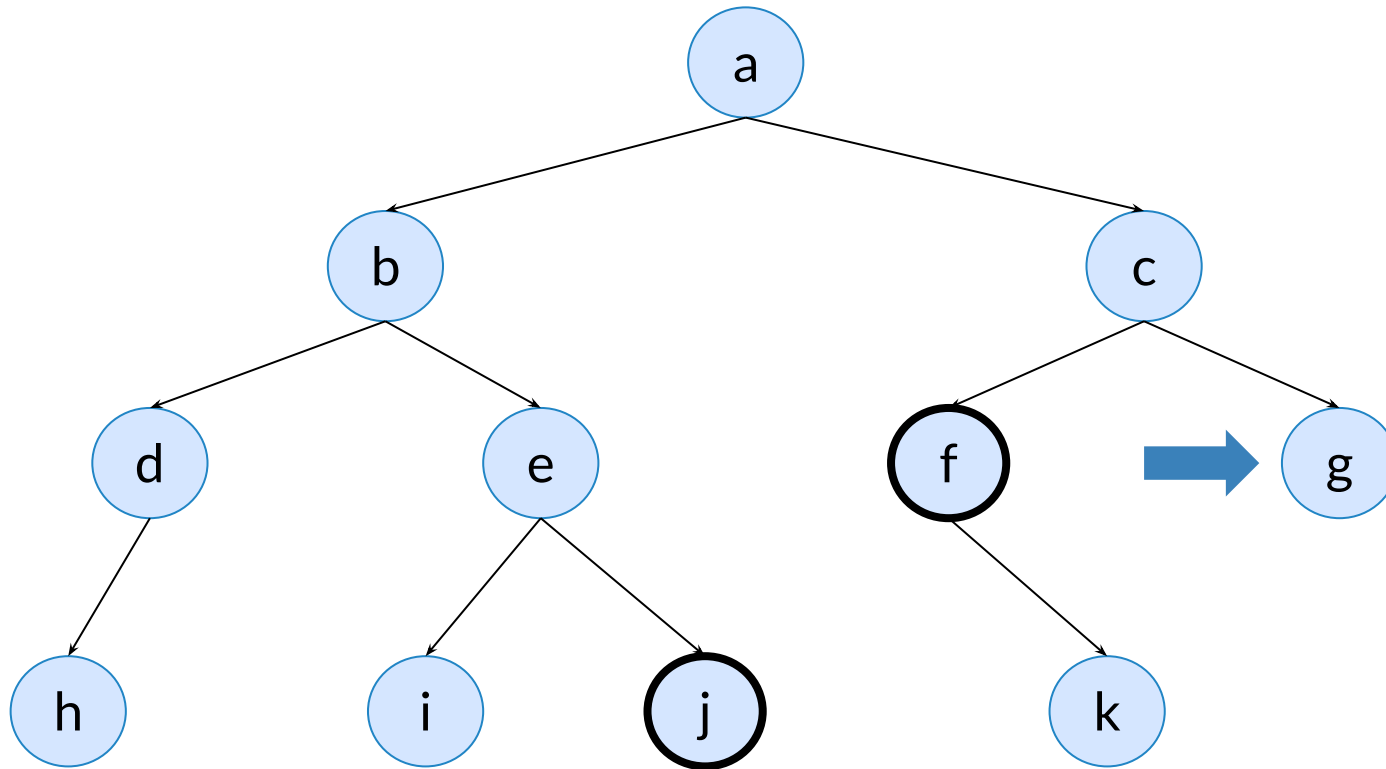
LIFO: f, g

Busca em Profundidade



LIFO: k, g

Busca em Profundidade



LIFO: g

Busca em Largura

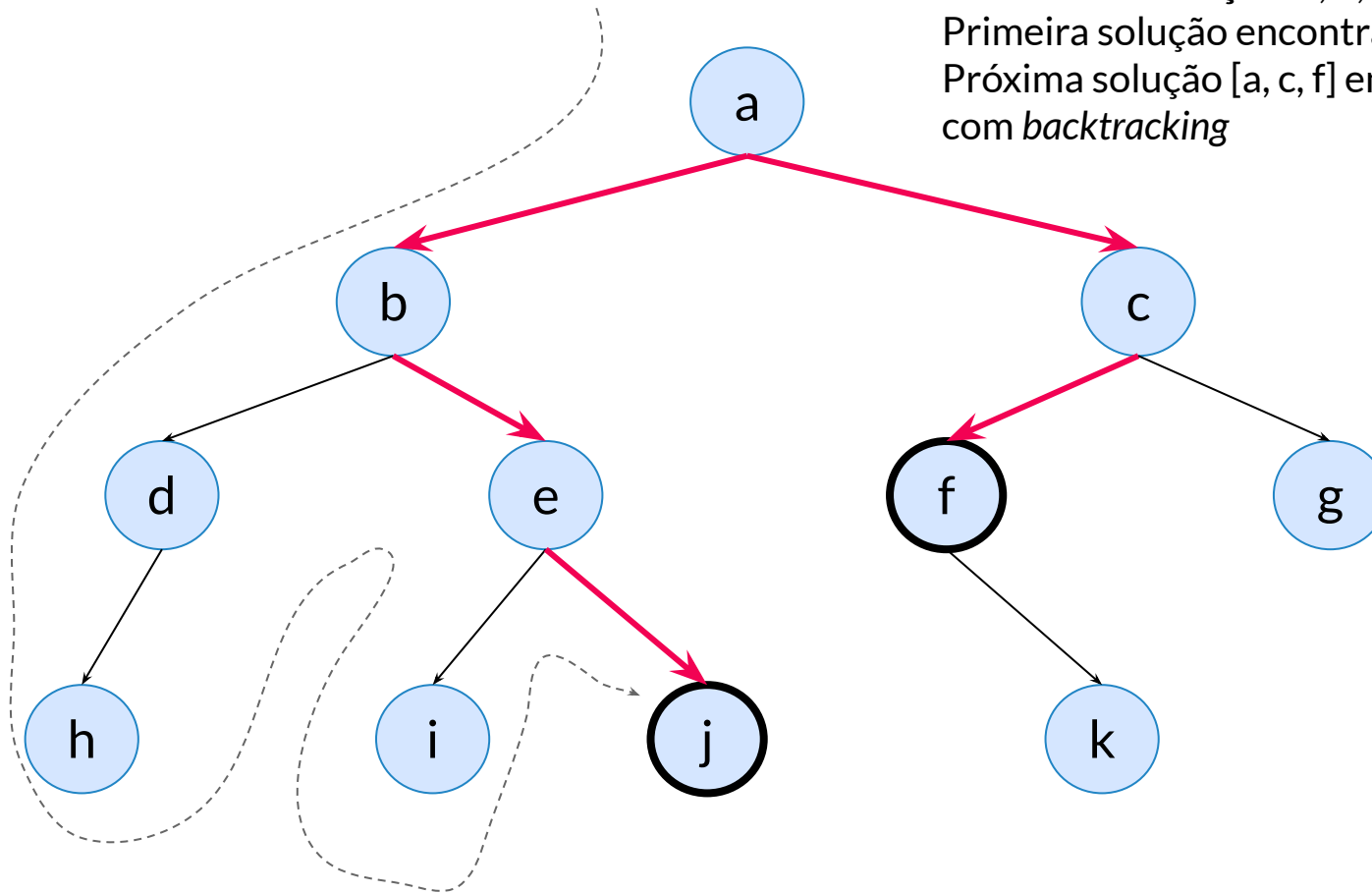
Estado inicial: a

Estados finais: j, f

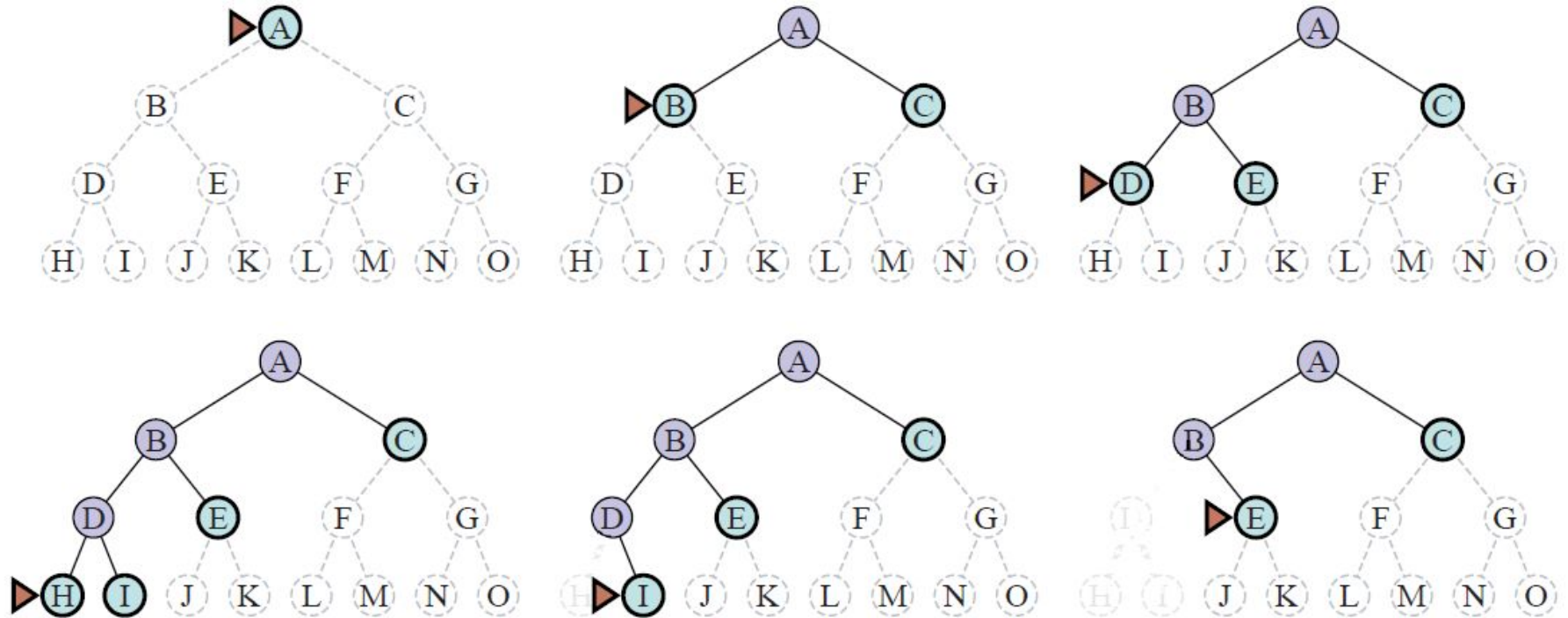
Ordem de visitação: a, b, d, h, e, i, j

Primeira solução encontrada [a, b, e, j]

Próxima solução [a, c, f] encontrada depois com *backtracking*



Busca em Profundidade – Resumo

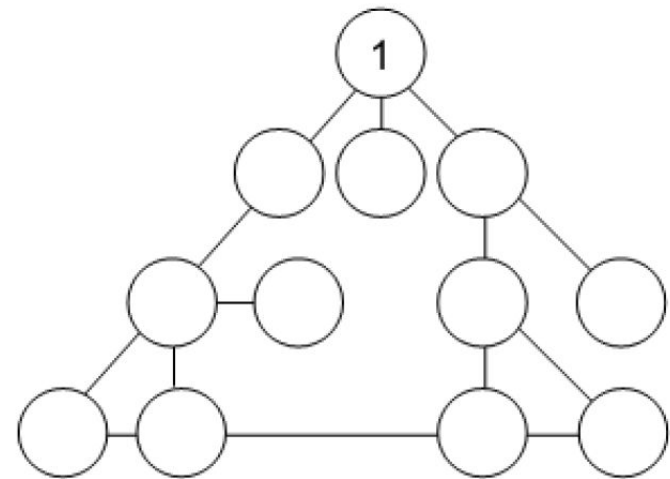
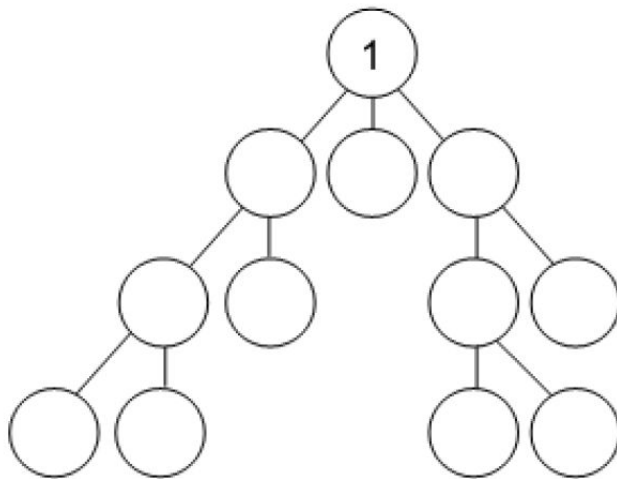


E assim continua...

Exercício #3

Indique a ordem na qual os nós são visitados na busca em profundidade

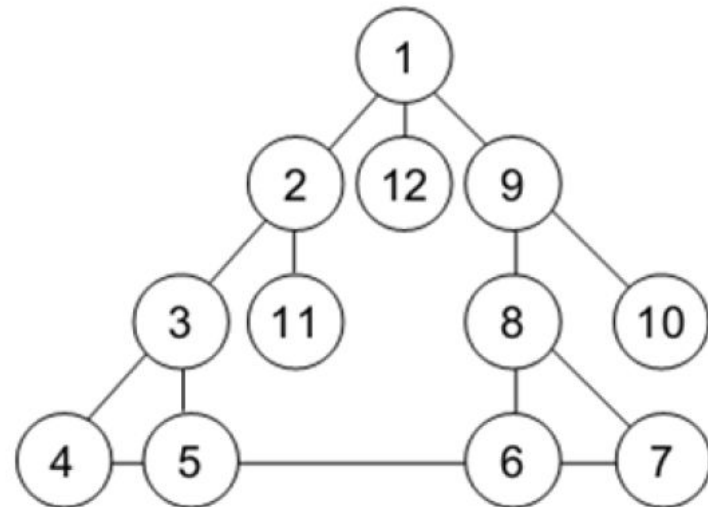
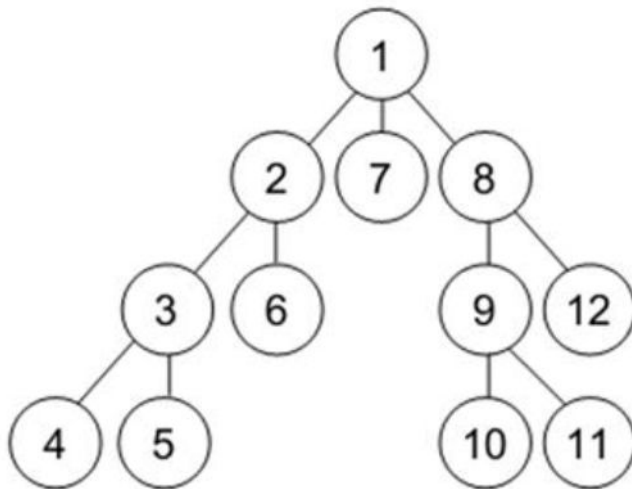
- Assuma que os sucessores são definidos da esquerda para a direita, e, depois, de cima para baixo



Exercício #3 – Solução

Indique a ordem na qual os nós são visitados na busca em profundidade

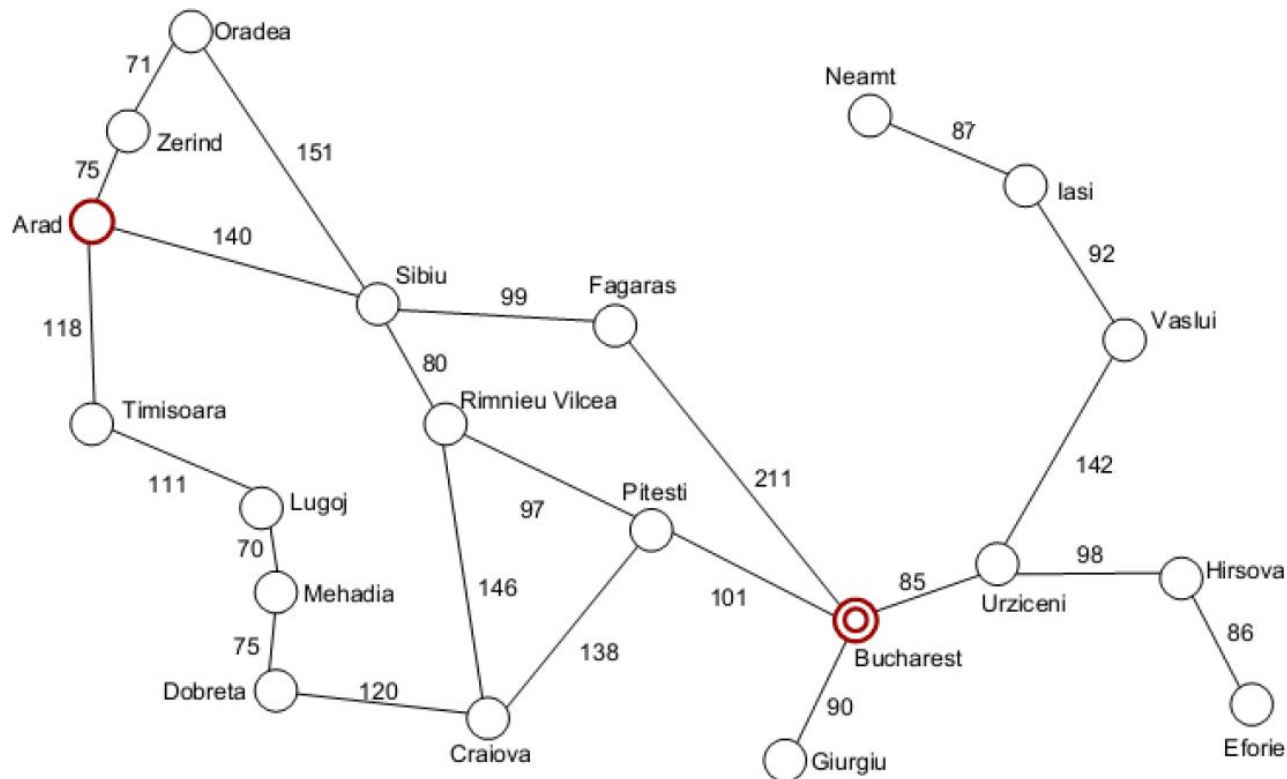
- Assuma que os sucessores são definidos da esquerda para a direita, e, depois, de cima para baixo



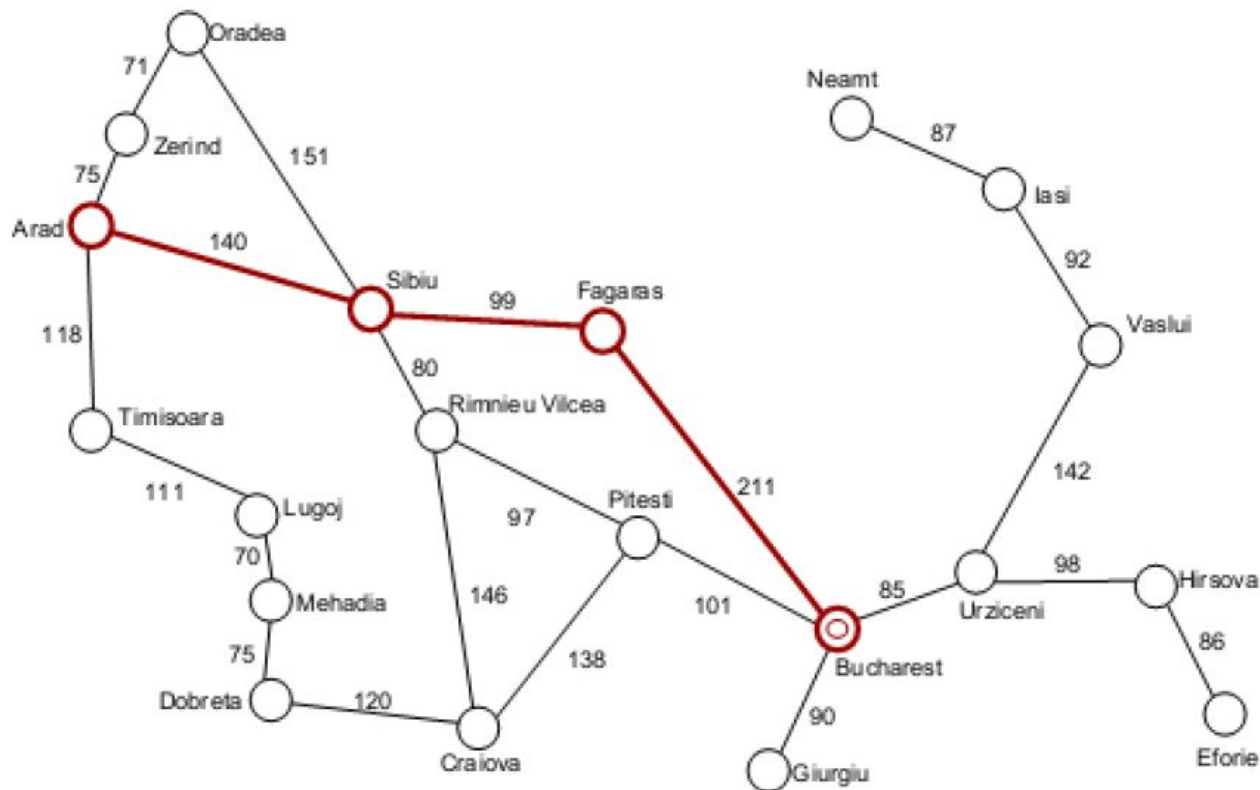
Exercício #4

Encontre o caminho e a árvore de busca de Arad até Bucareste usando a Busca em Profundidade

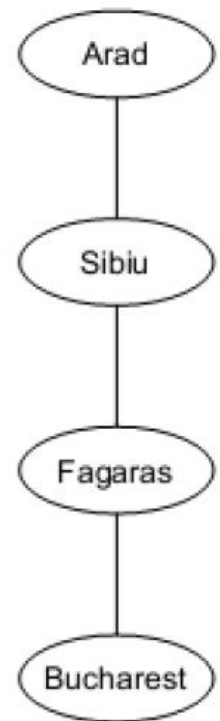
- Assuma que os sucessores de um nó são definidos em ordem alfabética



Exercício #4 – Solução



Árvore de Busca



Busca em Profundidade

Avaliação do algoritmo

- ▷ **Completo:** não (pode cair em caminhos infinitos ou *loops*)
- ▷ **Ótimo:** não
- ▷ **Complexidade**
 - Considerando *branch factor* b e profundidade máx. m
 - Tempo: $O(b^m)$ no pior caso
 - Espaço: $O(bm)$

Busca em Profundidade

Problema: Algoritmo pode se perder, caso o espaço de busca seja infinito

- Vai sempre em profundidade e pode chegar em um caminho infinito
- Pode nunca encontrar um nó final
- Refinamento → limitar a profundidade da busca

Busca com Profundidade Limitada

- ▷ Define uma **profundidade máxima L** , na qual a busca em profundidade deve parar
- ▷ Interessante quando se tem um conhecimento maior sobre o espaço de busca

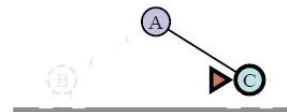
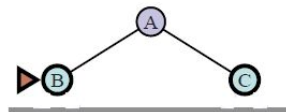
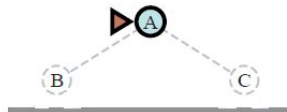
Aprofundamento Iterativo

Ideia → várias buscas em profundidade, aumentando o limite a cada iteração

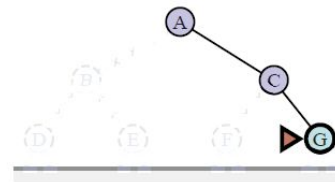
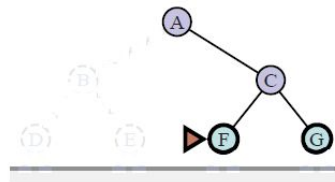
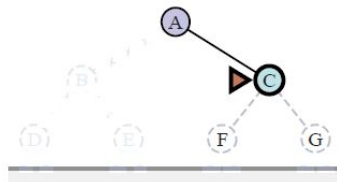
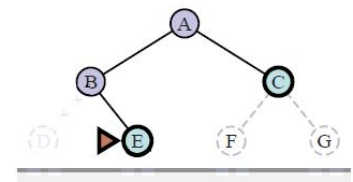
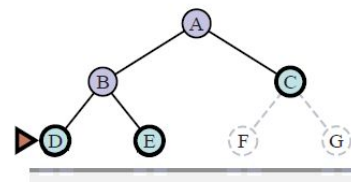
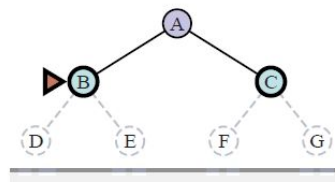
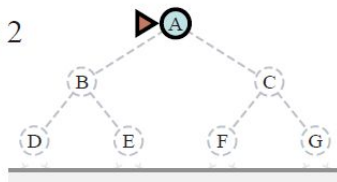
limit: 0



limit: 1

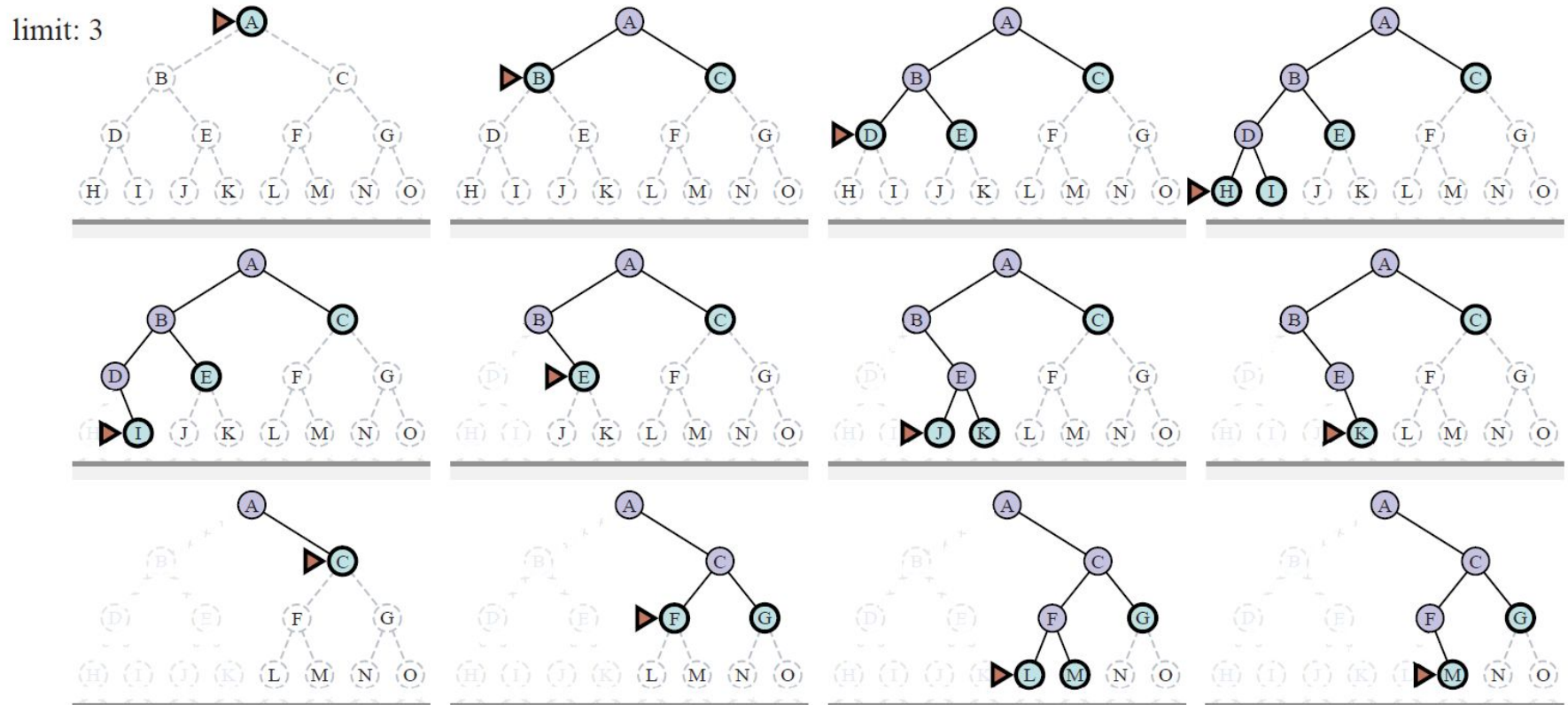


limit: 2



Aprofundamento Iterativo

Ideia → várias buscas em profundidade, aumentando o limite a cada iteração



Referências

RUSSELL, Stuart J.; NORVIG, Peter. **Artificial intelligence: a modern approach**. 4th ed. Pearson, 2021. (*Capítulo 3*)